Extending JMC for Real-World Concurrent Java Applications

Master's Thesis

Tanvi Moharir
RPTU Kaiserslautern

April 1, 2026

Supervisor: Prof. Rupak Majumdar
Max Planck Institute for Software Systems

Abstract

Testing large-scale concurrent programs remains a challenging problem, as traditional testing approaches do not prove effective in being able to find data races, deadlocks or assertion violations which may appear in certain interleavings of thread scheduling. Java Model Checker (JMC) is a general-purpose testing tool designed to test concurrent programs. JMC systematically explores thread interleavings by instrumenting Java bytecode. In this thesis, we extend JMC's capabilities, allowing us to test large industrial Java projects effectively. Specifically, we extend JMC to support key Java language and runtime features, including static initialization semantics, lambda expressions, and executor and future lifecycle management, native `Object` methods, enums, and interfaces. Additionally, we develop a custom JUnit test engine which enables JMC to integrate with any application seamlessly. We evaluate these extensions by applying JMC to Apache Iceberg, a production-grade distributed table format with complex code patterns. Our results demonstrate that these enhancements make JMC a practical tool for testing large-scale Java systems.

Contents

Abstract **1**

1 Introduction **6**
1.1 Our Contribution . . . 6
1.2 Outline . . . 7

2 Background **8**
2.1 Random Testing . . . 8
2.2 Java Threads and Bytecode . . . 8
2.3 JMC . . . 10
2.3.1 Execution Flow . . . 10
2.3.2 Core Components . . . 12
2.3.3 Illustrative Example: Detecting a Data Race with JMC . . . 14

3 Custom Test Engine **18**
3.1 The Problem: Verbose Programmatic Configuration . . . 18
3.2 Solution: JUnit 5 Platform Integration . . . 19
3.3 Annotation-Based Configuration . . . 19
3.4 Test Engine Architecture . . . 20
3.5 Service Provider Registration . . . 21

4 Static Initialization **22**
4.1 Static Initialization in Java . . . 22
4.1.1 Static Patterns in Java . . . 22
4.2 Challenges with Static . . . 23
4.3 Initial Solution: Extracting Static Initialization . . . 23
4.3.1 Transformation Strategy . . . 23
4.3.2 Runtime Reinitialization . . . 24
4.4 The Concurrent Access Problem . . . 24
4.4.1 The Race Condition . . . 25
4.5 Current Solution: Synchronized Static Initialization . . . 25
4.5.1 Three-Method Transformation . . . 25
4.5.2 Runtime Events . . . 26
4.5.3 The TrackStaticInit Tracker . . . 26

4.5.4 Execution Flow . . . 27
4.6 Nested Static Initialization . . . 28
4.6.1 The Nested Initialization Problem . . . 29
4.6.2 Depth Counter Solution . . . 29
4.6.3 Nested Initialization Flow . . . 30
4.7 Static Synchronized Methods . . . 31
4.7.1 The Static Synchronized Problem . . . 31
4.7.2 The NullPointerException Problem . . . 31
4.7.3 Initial Approach: Lock Registration in <clinit> . . . 31
4.7.4 Solution: Lazy Lock Registration . . . 32
4.8 Interface Static Fields . . . 33
4.9 Alternative Approaches Considered . . . 33
4.9.1 Class Reloading via URLClassLoader . . . 34
4.9.2 Class Unloading . . . 34
4.9.3 Reflection-Based Field Reset . . . 35

5 Lambdas **37**
5.0.1 The invokedynamic Instruction . . . 37
5.0.2 Bootstrap Method Handle . . . 37
5.0.3 Bootstrap Method Arguments . . . 38
5.1 The Instrumentation Challenge . . . 38
5.2 Common Lambda Patterns . . . 40
5.2.1 Executor Submissions . . . 40
5.2.2 CompletableFuture Chains . . . 41
5.2.3 Thread Constructors . . . 41
5.2.4 Parallel Streams . . . 41
5.3 Solution: Type Descriptor Rewriting . . . 41
5.3.1 Visitor Implementation . . . 42
5.3.2 Rewriting Strategy . . . 42

6 Executors and Futures **45**
6.1 Executors and Futures in Java . . . 45
6.1.1 ExecutorService and Thread Pools . . . 45
6.1.2 Future and Asynchronous Results . . . 46
6.1.3 ScheduledExecutorService . . . 46
6.2 The Instrumentation Challenge . . . 47
6.2.1 Initial State: Incomplete Implementation . . . 47
6.2.2 Type Mapping Challenges . . . 47
6.2.3 Interface vs. Concrete Implementation . . . 48
6.3 Common Executor Patterns . . . 48
6.3.1 Factory Method Usage . . . 48
6.3.2 Third-Party Library Integration . . . 48
6.3.3 Scheduled Task Execution . . . 49

6.3.4 Internal Wrapper Classes . . . 49
6.4 Solution: Hierarchical Type Mapping and Lifecycle Management . . . 49
6.4.1 Step 1: Enabling the Visitor Chain . . . 49
6.4.2 Step 2: Fixing JmcExecutors Type Mappings . . . 49
6.4.3 Step 3: Restructuring JmcExecutorService . . . 50
6.4.4 Step 4: Implementing Scheduled Executor Support . . . 50
6.4.5 Step 5: Visitor Changes for Type Mapping . . . 51
6.4.6 Step 6: Resource Management with TrackExecutors . . . 51

7 Object Class **54**
7.1 Object Class in Java . . . 54
7.1.1 Synchronization Methods . . . 54
7.1.2 Native Utility Methods . . . 55
7.2 The Runtime Challenge . . . 55
7.2.1 Cannot Replace Object Class . . . 55
7.2.2 Native Method Instrumentation . . . 55
7.3 Motivating Example: HashSet with Concurrent Access . . . 56
7.4 Attempted Solutions . . . 56
7.4.1 Initial Decision: No Instrumentation . . . 56
7.4.2 Attempted Solution 1: Clearing Access Flags . . . 56
7.4.3 Attempted Solution 2: JVM Flags to Disable Intrinsics . . . 56
7.4.4 Attempted Solution 3: Renaming Overridden Methods . . . 57
7.5 Current Solution: Delegation with Reflection . . . 58
7.5.1 Part 1: Wait/Notify/NotifyAll . . . 59
7.5.2 Part 2: Native Utility Methods . . . 60
7.5.3 Addressing the Infinite Read Problem . . . 62

8 Miscellaneous **66**
8.1 Enums . . . 66
8.1.1 The Challenge . . . 66
8.1.2 Solution: Skip Enum Instrumentation . . . 67
8.2 Final Fields . . . 67
8.2.1 The Challenge . . . 67
8.2.2 Solution: Ignore Final Fields . . . 68
8.3 Synchronized Annotation Preservation . . . 69
8.3.1 The Challenge . . . 70
8.3.2 Solution: Recursive Annotation Copying . . . 70
8.4 Concurrent Feature Detection . . . 70
8.4.1 Implementation . . . 70
8.5 Integration Fixes . . . 71
8.5.1 Listener.notify() False Positive . . . 71
8.5.2 Guava Joiner.join() vs Thread.join() . . . 72

9 Evaluation **76**
9.1 Synthetic Stress Tests 76
9.2 Testing Apache Iceberg with JMC 76
9.2.1 Targeting the Metadata Layer 77
9.2.2 Targeting Base Classes and Infrastructure 78
9.2.3 Catalog Implementations and CAS Mechanisms 78
9.3 Reproduction of Known Concurrency Issues 79

10 Conclusion **81**

Chapter 1

Introduction

Concurrent programs are difficult to test because bugs often depend on specific thread interleavings that are hard to reproduce. Manual testing only exercises a limited number of schedules, leaving many concurrency defects such as race conditions and atomicity violations latent even after extensive testing [58]. Unit testing frameworks face the same fundamental limitation: they validate only observed executions rather than the entire space of possible interleavings, and therefore cannot guarantee the absence of concurrency bugs [58]. Formal approaches such as specification-based model checking with TLA+ [57] can explore possible states automatically,but they operate on abstract specifications rather than concrete implementations, introducing a gap between the verified model and the actual program.

An alternative is to work directly at the implementation level. Several tools take this approach for concurrent Java programs. Java Pathfinder (JPF)[20] is a model checker that executes Java bytecode inside a custom virtual machine, systematically exploring thread interleavings to verify correctness properties. Lincheck[36] targets concurrent data structures by generating concurrent operation sequences and checking their results against a sequential specification using linearizability. Fray [11] is a concurrency testing framework that uses bytecode instrumentation to control thread scheduling, supporting randomized exploration strategies. JMC [25] follows a similar instrumentation-based approach but focuses on stateless model checking and random testing: it instruments compiled bytecode to insert yield points at all field accesses and then runs the program repeatedly under the adjusted scheduler that varies thread interleavings across iterations.

1.1 Our Contribution

This thesis extends JMC[1], a research-grade tool, to support testing of large-scale, real-world Java applications. Specifically, we enable JMC to test Apache Iceberg, an open-source distributed storage system with complex concurrency patterns.

First, we developed a custom JUnit 5 test engine that seamlessly integrates JMC into standard Java testing workflows through annotations, eliminating the need for verbose programmatic configuration. Second, we implemented comprehensive support for Java's static initialization semantics, handling static fields, methods, synchronized static blocks, interface fields, and nested

[1] https://jmc.mpi-sws.org/

static contexts in a way compatible with JMC's multi-iteration exploration model. Third, we extended JMC's instrumentation to support executors and futures, including custom implementations of `ScheduledExecutorService` and `ScheduledFuture`, along with their corresponding bytecode visitors. Fourth, we added support for common Java patterns essential for real-world applications: lambda expressions with `invokedynamic` instruction handling, enum types, and `Object` class methods, including native methods like `hashCode` and `equals`. Finally, we plug JMC to Apache Iceberg to demonstrate the effectiveness of the solutions mentioned above as being sufficient in testing the majority of patterns found in such systems. We faced several issues in integrating JMC with Apache Iceberg [2], starting from static executor pools that persist across test iterations and leak threads' native method calls (hashCode, equals etc). These issues helped us refine and strengthen our solutions. As part of the demonstration, we developed a test suite of around 50 test cases, which are focused on Iceberg's key features like schema evolution, row-level operations across multiple catalogs like InMemoryCatalog, JDBCCatalog, REST and Hadoop catalog which all internally work with different synchronization logic [54]. This demonstrates the robustness of the solutions we developed in being able to capture a wide range of patterns found in a production-grade system. We also reproduced three known concurrency issues in Iceberg's REST catalog [45, 44, 6].

These extensions transform JMC from a research prototype into a practical tool capable of systematically testing production-grade Java systems.

1.2 Outline

The remainder of this thesis is organized as follows. Chapter 2 provides background on random testing of concurrent programs, Java threads and bytecode, and introduces JMC's architecture and execution model. Chapters 3 through 8 present our technical contributions: the custom test engine (Chapter 3), static initialization handling (Chapter 4), lambda expression support (Chapter 5), executor and future implementations (Chapter 6), instrumentation of `Object` class methods (Chapter 7), along with enum and interface support and other miscellaneous enhancements (Chapter 8). Chapter 9 evaluates our approach through integration with Apache Iceberg, including benchmarks and observed metrics. Finally, Chapter 10 summarizes the key contributions and outlines the direction for future work.

Chapter 2

Background

This chapter provides the background necessary for understanding JMC's design and the extensions presented in this thesis. Section 2.1 introduces random testing and its application to concurrent programs. Section 2.2 covers Java threads and bytecode, the foundation on which JMC's instrumentation operates. Section 2.3 explains how JMC combines the ideas from previous sections along with a description of JMC's architecture, execution flow, and core components.

2.1 Random Testing

Random testing selects thread schedules in concurrent programs randomly rather than exploring them systematically. The intuition is that if a bug can be triggered by some schedule, executing the program repeatedly under randomized conditions increases the likelihood of encountering it. While this approach has been effective for uncovering edge cases in sequential programs [55], it is particularly useful for concurrency, where the number of possible thread interleavings grows exponentially with program complexity.

In concurrent settings, random testing introduces nondeterminism into the scheduler by varying the order in which threads execute at designated scheduling points. This enables exploration of diverse interleavings, including rare executions that may expose data races, atomicity violations, or deadlocks.

The effectiveness of this approach depends on both the placement and frequency of scheduling points. Sparse instrumentation limits the diversity of explored interleavings, while overly dense scheduling incurs significant overhead. In practice, bytecode-level field accesses provide a natural and effective granularity for introducing scheduling points in Java programs.

2.2 Java Threads and Bytecode

Java provides two primary mechanisms for writing concurrent programs. The first is direct thread management using `java.lang.Thread`: a developer creates a subclass or passes a `Runnable` to a `Thread` constructor, then calls `start()` to begin concurrent execution [51]. The second is the executor framework (`java.util.concurrent`), which abstracts thread management behind `ExecutorService` interfaces, allowing developers to submit tasks to thread pools without managing individual threads [9].

In both cases, the fundamental concurrency challenge is the same: multiple threads access shared mutable state, and the order of those accesses is determined by the JVM's thread scheduler at runtime [56]. Consider a simple example shown in Listing 2.1 where two threads increment a shared counter.

```
class Counter {
    int value = 0;

    void increment() {
        value++;
    }
}
```

Listing 2.1: Simple counter class with a non-atomic increment.

The `value++` operation appears atomic in source code, but the Java compiler translates it into multiple bytecode instructions as we see in Listing 2.2.

```
// increment()V
  ALOAD 0
  DUP
  GETFIELD Counter.value : I
  ICONST_1
  IADD
  PUTFIELD Counter.value : I
  RETURN
```

Listing 2.2: Bytecode for `Counter.increment()`.

The critical instructions are `GETFIELD` (which reads the current value of the field onto the operand stack) and `PUTFIELD` (which writes the computed result back to the field). Between these two instructions, another thread can execute its own `GETFIELD`, reading the same stale value. This is the classic lost-update race condition.

This bytecode-level view reveals something important: every shared field access in a Java program corresponds to a `GETFIELD` or `PUTFIELD` instruction (for instance fields) or `GETSTATIC`/`PUTSTATIC` (for static fields). These instructions are precisely the points where thread interleavings can produce different outcomes. JMC intercepts all such instructions to decide precisely the order in which threads execute instructions, allowing us to systematically explore all interleavings.

Random testing tools using bytecode instrumentation insert scheduling points at field accesses (e.g., `GETFIELD`, `PUTFIELD`) by rewriting compiled classes. These yield points allow a scheduler to control thread interleavings during execution. Combined with randomized scheduling across multiple runs, this approach explores different execution orders and increases the likelihood of exposing concurrency bugs.

2.3 JMC

JMC [1] is a general-purpose tool for verifying correctness of concurrent Java programs. JMC operates by systematically exploring different possible executions of a target program and checking assertions at the end of each execution. Unlike traditional testing that relies on the operating system's nondeterministic thread scheduler, JMC takes explicit control over thread scheduling to ensure comprehensive coverage of concurrent behaviors.

JMC's approach involves two key phases [28]: bytecode instrumentation and controlled execution. First, JMC instruments the compiled bytecode of the Java program to insert yield points at strategic locations. Then, during execution, JMC runs the instrumented program for multiple iterations under a custom runtime that imposes cooperative multithreading semantics. During each iteration, threads pause at read, write, and other relevant events, yielding control to JMC runtime, which decides which thread should execute next according to a pluggable scheduling strategy [28].

2.3.1 Execution Flow

Testing a program with JMC involves two distinct stages [28]: instrumentation of compiled bytecode and execution using JMC's custom runtime.

JMC defines a custom Java agent that instruments classes as they are loaded by the JVM [26]. The instrumentation process inserts calls to JMC runtime at strategic points to capture relevant events and pause threads. For example, the agent instruments every `GETFIELD` and `PUTFIELD` bytecode instruction to present read and write events to JMC runtime respectively.

The goal of instrumentation is to insert yield points throughout the target program. JMC achieves this using a Java agent that transforms classes during class loading. The transformation replaces traditional Java concurrency primitives with JMC-defined equivalents that interact specifically with JMC runtime to pass events and yield control when necessary. For instance, `java.lang.Thread` is replaced with `JmcThread`, `java.util.concurrent.locks.ReentrantLock` becomes `JmcReentrantLock`, and atomic classes like `AtomicInteger` are replaced with `JmcAtomicInteger`.

To demonstrate this transformation, consider the user program shown in Listing 2.3. The instrumented version produced by the JMC agent is presented in Listing 2.4.

Notice how, in Listing 2.4, `Thread` becomes `JmcThread`, `ReentrantLock` becomes `JmcReentrantLock`, and calls to `JmcRuntimeUtils.readEventWithoutYield()` and `JmcRuntime.yield()` are inserted at strategic points to enable the runtime to observe field accesses and control thread scheduling.

Once instrumented, JMC executes the instrumented code using a specified scheduling strategy. JMC runtime works alongside JVM runtime to control thread execution. While the JVM creates and manages actual OS threads as normal, JMC runtime ensures that only one application thread executes at any given moment, effectively serializing concurrent execution.

JMC runtime tracks all concurrent computations in the program threads, futures, executor

[1] https://github.com/mpi-sws-rse/jmc

```java
public class CounterRaceTest extends Thread {
    ReentrantLock lock;
    Counter counter;

    @Override
    public void run() {
        try {
            lock.lock();
            counter.increment();
        } finally {
            lock.unlock();
        }
    }
}
```

Listing 2.3: User program before instrumentation.

tasks, and other asynchronous operations referring to each as a task. When a task reaches a yield point (inserted during instrumentation), it blocks on a temporary synchronization primitive such as a CompletableFuture. This causes the JVM to pause that thread, preventing it from executing further.

Concurrently, JMC runtime maintains a separate scheduler thread that interacts with the `SchedulingStrategy` interface. Each time a task yields, the scheduler queries the strategy to determine which paused task should execute next. The scheduler then unblocks the selected task by completing its synchronization primitive, allowing that thread to resume execution. This process repeats at each yield point, with the scheduler controlling which thread runs at each step.

This architecture enables deterministic, sequential execution where only one application thread is active at a time. Although the program contains multiple threads, they execute in a controlled sequence rather than concurrently. By varying the scheduling decisions across multiple iterations of the same test, JMC systematically explores different orderings of operations, effectively exploring different interleavings that could occur under true concurrent execution. JMC's approach to serializing concurrent execution through controlled scheduling is depicted in Figure 2.1.

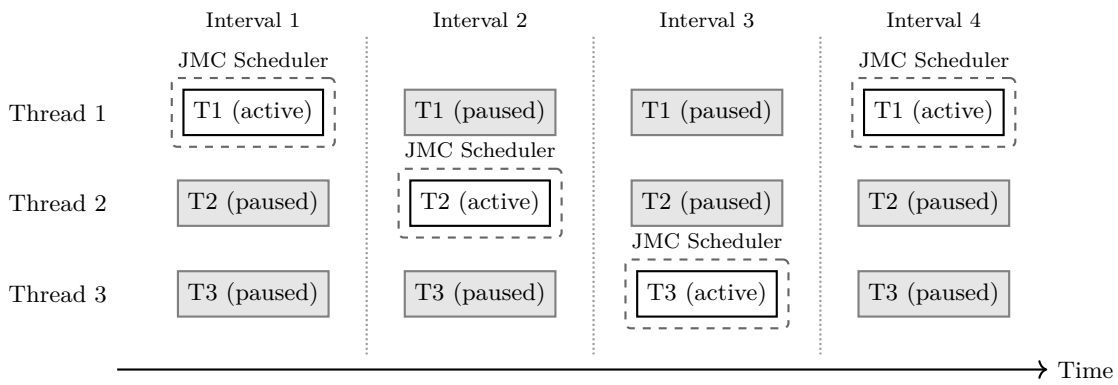

Figure 2.1: Sequential execution under JMC: only one task executes at a time while others remain paused. The JMC Scheduler controls which task runs at each interval.

```java
public class CounterRaceTest extends JmcThread {
    JmcReentrantLock lock;
    Counter counter;


    @Override
    public void run1() {
        try {
            JmcRuntimeUtils.readEventWithoutYield(...);
            JmcReentrantLock var10000 = this.lock;
            JmcRuntime.yield();
            var10000.lock();
            JmcRuntimeUtils.readEventWithoutYield(...);
            Counter var4 = this.counter;
            JmcRuntime.yield();
            var4.increment();
        } finally {
            JmcRuntimeUtils.readEventWithoutYield(...);
            JmcReentrantLock var5 = this.lock;
            JmcRuntime.yield();
            var5.unlock();
        }
    }
}
```

Listing 2.4: Program after JMC instrumentation.

The difference between normal JVM execution and execution under JMC is illustrated in Figure 2.2.

2.3.2 Core Components

JMC's architecture consists of three main components: the agent, the runtime, and the API.

Agent. Java provides a standard agent mechanism through the `-javaagent` command-line option [19], which allows code to run before the application's `main` method is invoked. An agent registers an instrumentation pipeline via the `java.lang.instrument` API [15], hooking into the JVM's class loading mechanism to transform bytecode as classes are loaded. This is a conventional approach widely used across the Java ecosystem: testing frameworks such as Mockito [39] use agents to generate mock classes at runtime, and server frameworks such as Spring [46] rely on agents for dynamic proxying and aspect-oriented programming. JMC follows the same pattern. The agent is responsible for bytecode instrumentation and is implemented in the `agent` subproject. Starting from the `InstrumentationAgent` [15] class, the agent uses the ASM [5] library to transform Java bytecode as classes are loaded. The agent accepts several configuration parameters passed via the `-javaagent` JVM flag:

- `debug`: Enables logging of instrumented classes for debugging purposes
- `debugSavePath`: Specifies the path to store instrumented class files (defaults to `build/generated/instrumented`)

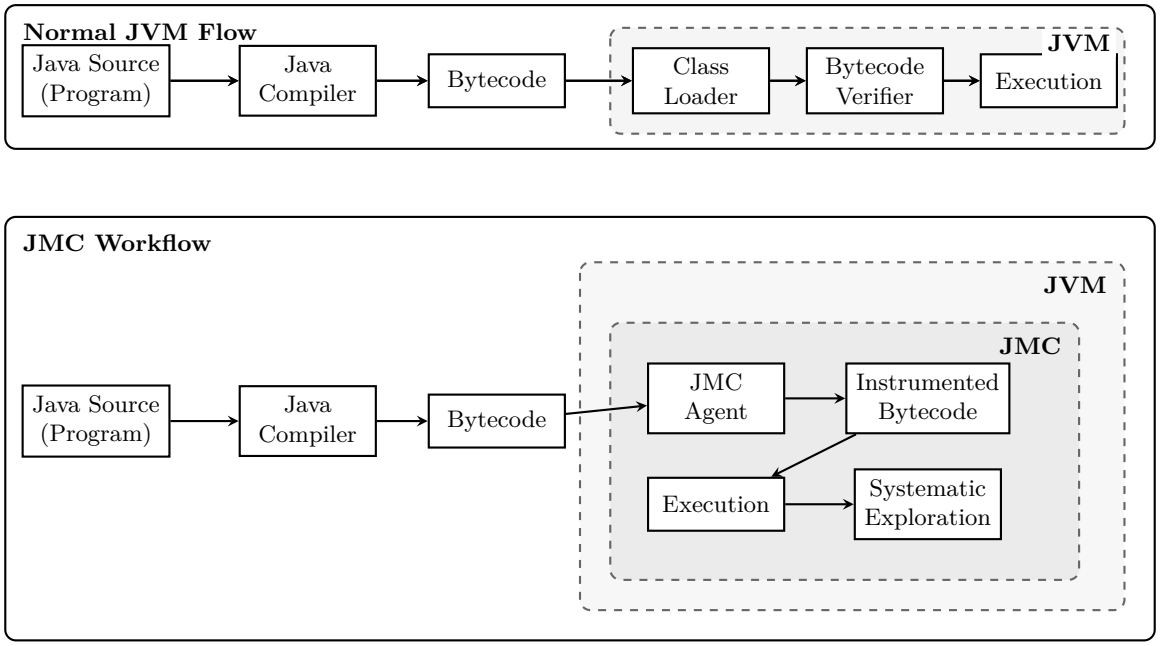

Figure 2.2: Comparison of normal JVM execution and execution under JMC.

- `instrumentingPackages`: A semicolon-separated list of package prefixes defining the scope of instrumentation. Only classes whose package names match these prefixes are instrumented
- `excludedPackages`: A semicolon-separated list of package prefixes to exclude from instrumentation
- `jmcRuntimeJarPath`: Path to the JMC library JAR file, whose classes are loaded first to ensure successful instrumentation (defaults to `/lib/jmc-<version>.jar`)

The agent invokes different ASM [5] visitors sequentially to instrument various concurrent primitives within the Java JDK. Each visitor targets specific bytecode patterns—for example, `JmcThreadVisitor` handles thread operations, `JmcSyncMethodVisitor` instruments synchronized blocks, and `JmcReadWriteVisitor` inserts yield points at field accesses.

An important design choice in JMC's agent is *lazy instrumentation*. Rather than scanning the classpath and transforming all classes upfront at agent startup, JMC registers a `ClassFileTransformer` via the standard `java.lang.instrument` API [15]. Concretely, the `InstrumentationAgent.premain()` method calls `inst.addTransformer(instrumentor, true)`, which causes the JVM to invoke the transformer's `transform()` method each time a class is loaded [15]. If a class is never loaded during a test run, it is never instrumented. This lazy approach is particularly beneficial for large codebases such as Apache Iceberg [3], where a single test may only exercise a small fraction of the thousands of available classes.

Runtime. The runtime component, implemented in the `org.mpi_sws.jmc.runtime` package serves as the central coordinator during model checking. Key classes include [27]:

- `JmcRuntime`: The main runtime class that orchestrates execution, manages iterations, and coordinates with scheduling strategies

- `JmcRuntimeUtils`: Utility methods for recording events such as reads, writes, and synchronization operations
- `TaskManager`: Manages the lifecycle of tasks (threads, futures, executor tasks), including pausing and resuming them based on scheduling decisions
- `ThreadPoolManager`: Tracks executor instances and their associated threads
- `JmcRuntimeEvent`: Defines the types of event that can be reported to the runtime (thread start, lock acquire, field read, etc.)

The runtime maintains state across multiple iterations, resetting the program state between iterations while preserving the information needed for systematic exploration. It also handles deadlock detection, timeout management, and trace recording to debug failed assertions.

API. The API [24] component provides JMC-controlled implementations of Java's concurrency primitives. These classes, located in the `org.mpi_sws.jmc.api.util.concurrent` package, include:

- Thread management: `JmcThread`, `JmcThreadFactory`
- Locks: `JmcReentrantLock`, `JmcLockSupport`
- Atomic variables: `JmcAtomicInteger`, `JmcAtomicBoolean`, `JmcAtomicReference`, `JmcAtomicReferenceArray`, `JmcAtomicStampedReference`
- Executors and futures: `JmcExecutors`, `JmcExecutorService`, `JmcFuture`, `JmcCompletableFuture`

These implementations extend or implement their corresponding Java standard library classes while adding hooks to interact with JMC runtime. When the agent instruments the bytecode, it replaces references to standard Java concurrency classes with these JMC equivalents, enabling the runtime to observe and control all concurrent operations.

2.3.3 Illustrative Example: Detecting a Data Race with JMC

To illustrate how JMC explores concurrent executions, we consider a simple case that contains a data race. The program in Listing 2.5 consists of a shared counter object whose value is incremented by two concurrent threads.

Intuitively, if two threads each increment a counter once, the final value of the counter should be 2.

$$\begin{array}{c} \text{Initial state: } \mathit{counter} = 0 \\ \text{Thread } T_1\text{: increment()} \\ \text{Thread } T_2\text{: increment()} \\ \hline \text{Expected final state: } \mathit{counter} = 2 \end{array}$$

However, because the increment operation is not atomic, the program contains a **data race**. The increment operation internally consists of three steps:

$$\texttt{value++} \equiv \begin{cases} \text{read value} \\ \text{add 1} \\ \text{write value} \end{cases}$$

```
class Counter {
    int value = 0;

    void increment() {
        value++;       // not atomic
    }
}

Thread T1: counter.increment()
Thread T2: counter.increment()

assert(counter.value == 2)
```

Listing 2.5: Counter with concurrent increment — pseudocode.

If the operations interleave sequentially, the correct result is produced:

$$\begin{aligned} T_1 &: \text{read } 0 \\ T_1 &: \text{write } 1 \\ T_2 &: \text{read } 1 \\ T_2 &: \text{write } 2 \end{aligned}$$

However, the following interleaving leads to an incorrect result:

$$\begin{aligned} T_1 &: \text{read } 0 \\ T_2 &: \text{read } 0 \\ T_1 &: \text{write } 1 \\ T_2 &: \text{write } 1 \end{aligned}$$

The final value becomes:

$$counter = 1$$

In a normal execution environment, thread scheduling is nondeterministic. The problematic interleaving occurs only under very specific scheduling conditions. Consequently, even running the program many times may not reveal the bug.

Instead of relying on random thread scheduling, JMC systematically explores all possible thread interleavings. To enable this exploration, the program is instrumented so that the model checker can observe and control relevant execution events.

Conceptually, the instrumented increment operation transforms to what can be seen in Listing 2.6.

```
read(value)
yield()
value = value + 1
write(value)
yield()
```

Listing 2.6: Instrumented increment with yield points.

The inserted `yield()` calls create scheduling points where JMC may switch between threads. When the program runs under JMC, the execution proceeds as illustrated in Figure 2.3.

Thread T_1	Thread T_2
read value (0)	
yield	
	read value (0)
	yield
write value (1)	
yield	
	write value (1)
	yield

Figure 2.3: Example interleaving explored by JMC

This schedule leads to the incorrect result $counter = 1$, causing the assertion to fail. Because JMC explores executions systematically, it is guaranteed that it will discover such problematic schedules.

The instrumentation introduced by JMC serves several purposes:

- **Event tracking**: reads and writes to shared variables are recorded.
- **Scheduling control**: yield points allow the scheduler to switch threads.
- **Execution monitoring**: assertions are checked across all explored schedules.

For example, a simplified fragment of the instrumented code includes calls of the form seen in Listing 2.7

```
JmcRuntimeUtils.readEvent(...)
JmcRuntimeUtils.writeEvent(...)
JmcRuntime.yield()
```

Listing 2.7: JMC runtime calls for event tracking.

These runtime calls allow JMC to observe memory accesses and control the execution order of threads.

Execution architecture. At a high level, the execution flow under JMC proceeds as follows:

1. The original program is instrumented by the JMC agent to insert yield points at concurrency-relevant operations.

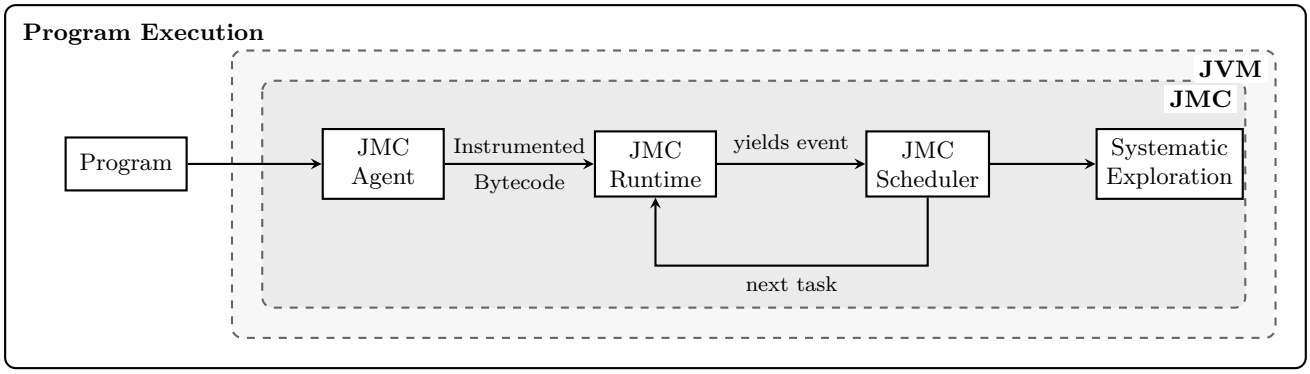

Figure 2.4: JMC execution architecture showing the flow from program to systematic exploration.

2. The instrumented program executes under JMC runtime, which tracks all tasks (threads, futures, executor tasks).
3. When a task reaches a yield point, it is blocked, and the runtime notifies the scheduler.
4. The scheduler (implementing a `SchedulingStrategy`) decides which paused task should execute next.
5. The runtime unblocks the selected task, allowing it to continue execution sequentially.
6. This process repeats at each yield point, with only one task executing at a time.
7. Across multiple iterations, different scheduling decisions explore different interleavings of operations.
8. If an assertion violation or deadlock occurs, JMC reports the execution trace that led to the failure.

The key insight is that JMC serializes concurrent execution: although multiple threads exist in the JVM, only one application thread executes at any moment, with the scheduler controlling which thread runs at each step. This deterministic, sequential execution enables systematic exploration of interleavings that would occur nondeterministically under true concurrent execution.

Chapter 3

Custom Test Engine

Integrating model checking tools into existing development workflows is crucial for practical adoption. Initially, JMC required developers to write verbose programmatic code to configure and execute tests. To address this limitation, we developed a custom JUnit 5 test engine [31] that enables annotation-based testing, allowing JMC to integrate seamlessly with standard Java testing infrastructure.

3.1 The Problem: Verbose Programmatic Configuration

Before the custom test engine, running JMC tests required explicit instantiation of several components: the model checker itself, its configuration, and a test target wrapper. Consider testing the counterexample from Chapter 2. The test for this was very verbose as shown in Listing 3.1.

```java
@Test
public void testCounterRace() throws JmcCheckerException {
    JmcCheckerConfiguration config =
        new JmcCheckerConfiguration.Builder()
            .numIterations(100)
            .debug(true)
            .build();

    JmcModelChecker jmcModelChecker = new JmcModelChecker(config);

    JmcTestTarget target =
        new JmcFunctionalTestTarget(
            "CounterRaceTest",
            () -> {
                // Test logic here
                ...
            });

    jmcModelChecker.check(target);
}
```

Listing 3.1: JMC test with verbose programmatic configuration.

This approach has several drawbacks. First, it requires substantial boilerplate code for

each test, obscuring the actual test logic. Second, it does not integrate with standard testing frameworks, meaning IDE test runners and build tools do not recognize these as regular tests. Third, configuration must be repeated across tests or extracted into helper methods, leading to code duplication. Finally, developers must learn JMC-specific APIs rather than using familiar testing patterns.

3.2 Solution: JUnit 5 Platform Integration

JUnit 5 [31] provides a modular architecture that separates the test platform from test engines [30]. The platform provides infrastructure for discovering and executing tests, while engines implement the actual test execution logic. This architecture allows third-party tools to integrate by implementing the `TestEngine` interface.

We developed `JmcTestEngine`, a custom test engine that integrates JMC into the JUnit 5 platform [31]. This engine discovers tests annotated with JMC-specific annotations and executes them using the JMC model checker. The engine is registered with the JUnit platform using Java's ServiceLoader mechanism, which automatically discovers and loads the engine at runtime. Now our test looks much cleaner, as seen in Listing 3.2.

```
@JmcCheck
@JmcCheckConfiguration(numIterations = 100, debug = true)
public void testCounterRace() {
    ...
}
```

Listing 3.2: Equivalent JMC test with annotation-based configuration.

The test method contains only the test logic, with configuration specified declaratively through annotations. All boilerplate code is eliminated, and the test integrates seamlessly with the existing JUnit infrastructure.

3.3 Annotation-Based Configuration

The custom test engine introduces two primary annotations that enable a declarative test configuration [23].

The `@JmcCheck` Annotation. This annotation marks a method or class as a JMC test. It serves as a marker that the test engine uses during discovery to identify which tests should be executed under JMC's control. The annotation is marked with JUnit's `@Testable` meta-annotation, ensuring that the JUnit platform recognizes it as a valid test annotation [23].

The `@JmcCheckConfiguration` Annotation. This annotation provides configuration parameters for JMC tests. It can be applied at the method level for the individual test configuration or at the class level to provide the default configuration for all tests in the class. The annotation supports several parameters [23]:

- `strategy`: Specifies which scheduling strategy to use (e.g., `"random"` for random exploration, `"trust"` for exhaustive enumeration)
- `numIterations`: The number of iterations to explore
- `debug`: Enables debug logging and additional diagnostic information
- `reportPath`: Specifies where the JMC report should be saved
- `seed`: Provides a random seed for reproducible exploration

Configuration follows a hierarchical resolution strategy: method-level annotations override class-level annotations, which in turn override default values. This allows developers to specify common configuration at the class level while customizing individual tests as needed [23].

3.4 Test Engine Architecture

The custom test engine in JUnit [31] can be understood by two-phase execution model: discovery and execution. Figure 3.1 illustrates these phases.

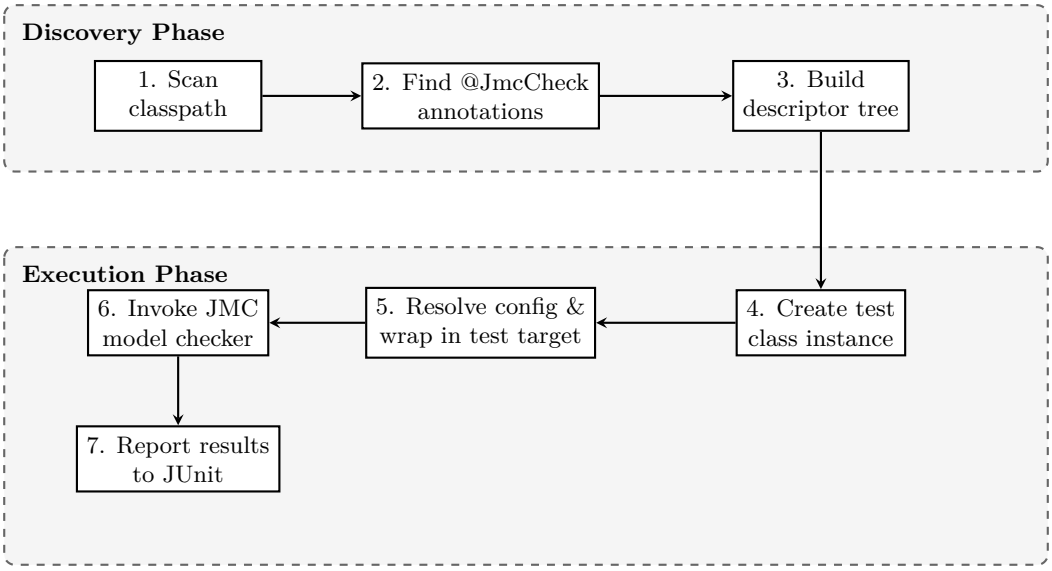

Figure 3.1: JMC test engine phases: discovery identifies tests, execution runs them through the model checker.

Discovery Phase. During discovery, the test engine scans the classpath to identify JMC tests. The JUnit platform provides several selector types that specify what to discover: `ClasspathRootSelector` for scanning entire classpath roots, `PackageSelector` for specific packages, `ClassSelector` for individual classes, and `MethodSelector` for specific methods [31].

For each selector, the engine uses reflection to find classes and methods annotated with `@JmcCheck` or `@JmcCheckConfiguration`. The discovery process builds a hierarchical tree of test descriptors. At the root is `JmcEngineDescriptor`, representing the entire test engine. Each discovered test class becomes a `JmcClassTestDescriptor`, and each test method within a class becomes a `JmcMethodTestDescriptor`.

Execution Phase. During execution, the test engine traverses the descriptor tree and executes each test [31]. For each `JmcMethodTestDescriptor`, the engine performs several steps:

First, it creates an instance of the test class using reflection. This instance is used to invoke the test method, following standard JUnit conventions.

Second, it resolves the configuration for the test. The engine checks for `@JmcCheckConfiguration` annotations at both the method and class levels, applying the hierarchical resolution strategy. If no annotations are present, default values are used.

Third, it wraps the test method in a `JmcFunctionalTestTarget`. This wrapper is a lambda-based abstraction that JMC's model checker can invoke repeatedly across different iterations. The wrapper handles method invocation and exception propagation.

Fourth, it creates a `JmcModelChecker` instance with the resolved configuration and invokes its `check()` method with the test target. The model checker then performs systematic exploration as described in Chapter 2.

Finally, it reports the results back to the JUnit platform. If the model checker finds a bug (assertion failure, deadlock, etc.), the test is marked as failed with the appropriate exception. Otherwise, the test is marked as successful.

3.5 Service Provider Registration

The custom test engine is registered [43] with the JUnit platform using Java's ServiceLoader mechanism. A service provider configuration file is placed at
`META-INF/services/org.junit.platform.engine.TestEngine`
containing the fully qualified name of the engine class:
`org.mpi_sws.jmc.integrations.junit5.engine.JmcTestEngine`.

When the JUnit platform starts, it uses ServiceLoader to discover all available test engines. The platform automatically loads and initializes the JMC test engine, making it available for test discovery and execution [31]. This mechanism requires no explicit configuration from users; simply including the JMC library in the classpath is sufficient for the engine to be recognized.

Chapter 4

Static Initialization

Static initialization in Java is a fundamental mechanism that ensures classes are properly initialized before use [47]. However, this mechanism presents unique challenges for tools like JMC, which require the ability to reset program state across multiple iterations. This chapter describes JMC's approach to handling static initialization, from the initial solution through the problems encountered with concurrent access, to the final solution involving synchronized tracking and nested initialization support.

4.1 Static Initialization in Java

The Java Virtual Machine [32] dynamically manages classes and interfaces through three phases: loading, linking, and initialization. During loading, the JVM locates the binary representation of a class (typically a `.class` file) and creates an internal representation of it. Linking then integrates the class into the runtime by verifying its correctness, allocating memory for static fields, and resolving symbolic references into direct ones. Finally, initialization executes the class initializer method `<clinit>`, which assigns explicit values to static fields and runs static blocks. These phases may be triggered lazily during execution, allowing the JVM to load and prepare classes on demand while maintaining type safety and consistency across different class loaders [32].

The `<clinit>` method is implicitly synchronized by the JVM [7]. When multiple threads attempt to access a class for the first time, only one thread executes `<clinit>` while other threads block until initialization completes. This guarantee prevents race conditions during class loading and ensures that static fields are initialized exactly once.

4.1.1 Static Patterns in Java

Java supports several static patterns, each with distinct initialization semantics [47]:

Static fields belong to the class rather than instances. They can be mutable or final, and are initialized when the class is loaded [47]. Consider the example shown in Listing 4.1.

Static methods can be called without creating an instance of the class. They can only access static fields and other static methods [47]. This can be seen in sample code in Listing 4.2.

Java interfaces, as we can see in Listing 4.3, can declare static fields, which are implicitly `public static final`. These fields are initialized when the interface is loaded [47].

```java
class Example {
    static int counter = 0;
    static final String NAME = "Example";
}
```

Listing 4.1: Static field declarations.

```java
class MathUtils {
    static int add(int a, int b) {
        return a + b;
    }
}
```

Listing 4.2: Static method example.

Static methods can be declared synchronized, as seen in Listing 4.4 which means they acquire the class-level lock (the `Class` object's monitor) before executing [47].

Static fields can reference other classes' static fields, creating initialization dependencies. This is visible in the example from Listing 4.5.

When `B` is initialized, accessing `A.x` triggers initialization of `A` first. The JVM handles these dependencies correctly, ensuring proper initialization order.

4.2 Challenges with Static

JMC's requirement to explore multiple iterations with fresh state conflicts fundamentally with the JVM's "initialize once" guarantee [7]. After the first iteration initializes a class's static fields, subsequent iterations see the already-initialized values rather than starting fresh. This breaks the independence of iterations and can mask bugs or create false positives.

Let us consider 'Counter' example again as shown in Listing 4.6.

In the first iteration, `Counter.value` starts at 0. If the test increments it to 5, the second iteration begins with `value = 5` rather than 0. This accumulated state across iterations makes it impossible to explore independent execution paths.

4.3 Initial Solution: Extracting Static Initialization

The initial approach to solving this problem involved bytecode transformation to extract the static initialization logic into a separate method that could be called multiple times.

4.3.1 Transformation Strategy

The JMC agent transformed classes with static initializers by extracting the `<clinit>` method body into a new method called `$staticInit`. The original `<clinit>` would then register the class with the JMC runtime and call `$staticInit`. The `ComplexStaticInit` class can be seen in

After instrumentation, the transformation is visible in Listing 4.8.

```java
interface Constants {
    int MAX_SIZE = 100;  // implicitly public static final
}
```

Listing 4.3: Interface static field.

```java
class Singleton {
    private static Singleton instance;

    static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Listing 4.4: Static synchronized method.

4.3.2 Runtime Reinitialization

The JMC runtime maintained a registry of all classes with static initializers. The `registerStaticInitializedClass` method added each class to this registry during first-time initialization.

At the start of each iteration, the runtime's `initIteration` method called `invokeStaticInitializedClasses`, which used reflection to invoke `$staticInit` on each registered class, as seen in Listing 4.9.

```java
public static void invokeStaticInitializedClasses(int iteration) {
    for (Class<?> clazz : staticInitializedClassesList) {
        Method m = clazz.getDeclaredMethod("$staticInit");
        m.setAccessible(true);
        m.invoke(null);  // Static method, no instance needed
    }
}
```

Listing 4.9: Runtime reinitialization via reflection.

This approach successfully reset static fields across iterations, allowing JMC to explore independent execution paths.

4.4 The Concurrent Access Problem

The initial solution worked correctly for single-threaded scenarios but failed when multiple threads accessed a class simultaneously. The problem stemmed from a critical oversight: the extracted `$staticInit` method was not synchronized, while the original `<clinit>` was implicitly synchronized by the JVM [7].

```java
class A {
    static int x = 3;
}

class B {
    static int y = A.x;  // Triggers initialization of A
}
```

Listing 4.5: Nested static initialization dependency.

```java
class Counter {
    static int value = 0;

    static void increment() {
        value++;
    }
}
```

Listing 4.6: Static counter with accumulated state across iterations.

4.4.1 The Race Condition

Consider a test where multiple threads access a class with static initialization. This is demonstrated in Listing 4.10.

After instrumentation, the transformed `ComplexStaticInit` is visible in Listing 4.11.

The JVM's implicit synchronization of `<clinit>` would have prevented this race [7], but the extracted `$staticInit` lacks this protection.

4.5 Current Solution: Synchronized Static Initialization

The solution required mimicking the JVM's synchronization semantics while maintaining JMC's ability to reset static state across iterations. This was achieved through a combination of bytecode transformation, runtime events, and a specialized tracker.

4.5.1 Three-Method Transformation

The new transformation creates three methods instead of two:

1. `$staticInitBody`: Contains the actual initialization logic (private)
2. `$staticInitImplicit`: Called by `<clinit>` during first-time class loading, wraps `$staticInitBody` with synchronization events
3. `$staticInitExplicit`: Called by JMC runtime for reinitialization, calls `$staticInitBody` directly

The transformed `ComplexStaticInit` class is visible in Listing 4.12.

```java
class ComplexStaticInit {
    static int BASE_VALUE;
    static int COMPUTED_VALUE;

    static {
        BASE_VALUE = 10;
        COMPUTED_VALUE = BASE_VALUE + 32;
    }
}
```

Listing 4.7: Class with static initializer block.

```java
static class ComplexStaticInit {
    static int BASE_VALUE;
    static int COMPUTED_VALUE;

    private static void $staticInit() {
        BASE_VALUE = 10;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();

        int var10000 = BASE_VALUE;
        JmcRuntimeUtils.readEventWithoutYield(...);
        JmcRuntime.yield();

        COMPUTED_VALUE = var10000 + 32;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();
    }

    static {
        JmcRuntimeUtils.registerStaticInitializedClass(
            ComplexStaticInit.class);
        $staticInit();
    }
}
```

Listing 4.8: Instrumented class with extracted `$staticInit` method.

4.5.2 Runtime Events

Two new runtime events coordinate static initialization:

`START_STATIC_INIT_EVENT`: Fired when a thread begins static initialization. This event signals to the tracker that the current task wants to execute static initialization.

`END_STATIC_INIT_EVENT`: Fired when static initialization completes. This event signals that the current task has finished and another waiting task can proceed.

These events are fired using "without yield" variants (`startStaticInitEventWithoutYield` and `endStaticInitEventWithoutYield`) to avoid triggering scheduling decisions during the JVM's own class loading process.

4.5.3 The TrackStaticInit Tracker

The `TrackStaticInit` tracker implements the synchronization logic. It maintains:

```java
// Test (pseudocode)
ExecutorService executor = Executors.newFixedThreadPool(3);
for (int i = 0; i < 3; i++) {
    executor.submit(() -> {
        int value = ComplexStaticInit.COMPUTED_VALUE;
        // Use value
    });
}

// Class being accessed
class ComplexStaticInit {
    static int BASE_VALUE;
    static int COMPUTED_VALUE;

    static {
        BASE_VALUE = 10;
        COMPUTED_VALUE = BASE_VALUE + 32;  // Should be 42
    }
}
```

Listing 4.10: Concurrent access to static initialization.

- `currentStaticInitTask`: A map from task ID to initialization depth (for nested static initialization)
- `activeTasks`: The set of all active (non-blocked) tasks

When a task fires `START_STATIC_INIT_EVENT`:

1. If the task already has an entry in `currentStaticInitTask`, increment its depth counter (nested initialization)
2. Otherwise, add the task with depth 1

When a task fires `END_STATIC_INIT_EVENT`:

1. Decrement the task's depth counter
2. If the counter reaches 0, remove the task from `currentStaticInitTask`

The tracker's `updateEvent` method returns the set of active tasks. Crucially, if `currentStaticInitTask` is non-empty, it returns only the tasks currently performing static initialization. This ensures that only one task can execute static initialization at a time, while other tasks remain blocked.

4.5.4 Execution Flow

When three threads access `ComplexStaticInit.COMPUTED_VALUE` under the new solution, the tracker ensures only one thread initializes at a time. Figure 4.1 illustrates this synchronized execution flow.

This mechanism ensures only one thread executes static initialization at a time, matching the JVM's semantics.

```java
class ComplexStaticInit {
    static int BASE_VALUE;
    static int COMPUTED_VALUE;

    private static void $staticInit() {
        BASE_VALUE = 10;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();  // YIELD POINT 1

        int temp = BASE_VALUE;
        JmcRuntimeUtils.readEventWithoutYield(...);
        JmcRuntime.yield();  // YIELD POINT 2

        COMPUTED_VALUE = temp + 32;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();  // YIELD POINT 3
    }

    static {
        JmcRuntimeUtils.registerStaticInitializedClass(...);
        $staticInit();
    }
}
```

Listing 4.11: Instrumented class with unsynchronized `$staticInit` — race condition.

4.6 Nested Static Initialization

The synchronized solution resolved the concurrent access problem but revealed another issue: nested static initialization. When one class's static initializer accesses another class's static field, both classes must be initialized, potentially creating a nested initialization scenario.

```java
class ComplexStaticInit {
    static int BASE_VALUE;
    static int COMPUTED_VALUE;

    private static void $staticInitBody() {
        BASE_VALUE = 10;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();

        int temp = BASE_VALUE;
        JmcRuntimeUtils.readEventWithoutYield(...);
        JmcRuntime.yield();

        COMPUTED_VALUE = temp + 32;
        JmcRuntimeUtils.writeEventWithoutYield(...);
        JmcRuntime.yield();
    }

    public static void $staticInitExplicit() {
        $staticInitBody();
    }

    public static void $staticInitImplicit() {
        JmcRuntimeUtils.startStaticInitEventWithoutYield();
        $staticInitBody();
        JmcRuntimeUtils.endStaticInitEventWithoutYield();
    }

    static {
        JmcRuntimeUtils.registerStaticInitializedClass(...);
        $staticInitImplicit();
    }
}
```

Listing 4.12: Three-method transformation with synchronized initialization.

4.6.1 The Nested Initialization Problem

Listing 4.13 demonstrates the nested case. When Thread 1 initializes `B`, Thread 1 fires `START_STATIC_INIT_EVENT` for B. Tracker adds Thread 1 to `currentStaticInitTask` with depth 1. Thread 1 begins executing B's `$staticInitBody`. B's initialization accesses `A.x`, triggering A's initialization. Thread 1 fires `START_STATIC_INIT_EVENT` for A. With the original tracker logic, this would block Thread 1 (it's already in `currentStaticInitTask`). Deadlock: Thread 1 is waiting for itself to finish

The problem is that the same thread needs to perform nested static initialization, but the tracker treats all static initialization as a single critical section.

4.6.2 Depth Counter Solution

The solution uses a depth counter to track nested static initialization. Instead of storing just a boolean or single value in `currentStaticInitTask`, the tracker stores a map, like Listing 4.14, from task ID to initialization depth.

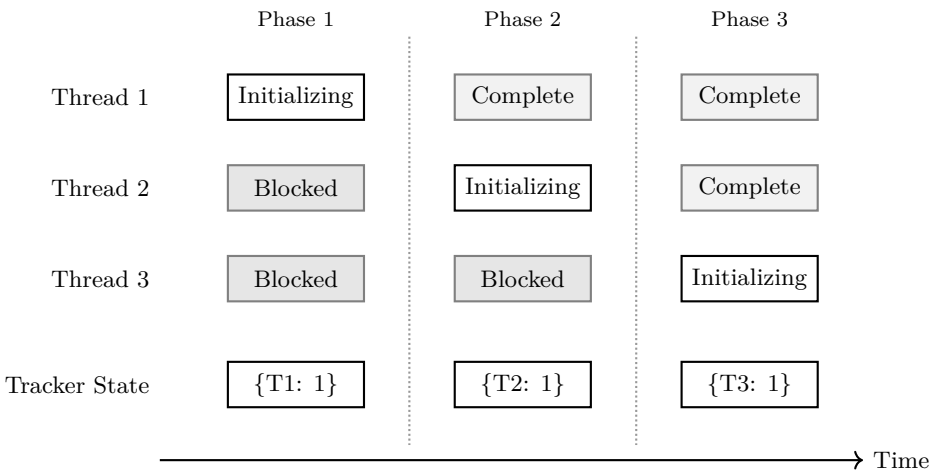

Figure 4.1: Synchronized static initialization: only one thread initializes at a time. The tracker maintains a map of active initializing threads with their depth counters.

```
// Test (pseudocode)
ExecutorService executor = Executors.newFixedThreadPool(3);
for (int i = 0; i < 3; i++) {
    executor.submit(() -> {
        int value1 = ComplexStaticInit.COMPUTED_VALUE;
        int value2 = B.y;  // Triggers nested initialization
    });
}

// Classes with nested dependencies
class A {
    static int x = 3;
}

class B {
    static int y = A.x;  // Accessing A.x triggers A's init
}
```

Listing 4.13: Nested static initialization test scenario.

Listing 4.15 shows when a task fires `START_STATIC_INIT_EVENT` and Listing 4.16 shows when a task fires `END_STATIC_INIT_EVENT`.

The tracker's `updateEvent` method returns active tasks based on the state of `currentStaticInitTask`, as seen in Listing 4.17.

4.6.3 Nested Initialization Flow

With the depth counter, Thread 1's nested initialization proceeds correctly. Figure 4.2 illustrates how the depth counter tracks nested initialization.

This approach allows the same thread to perform nested static initialization while still preventing multiple threads from initializing concurrently.

```java
private final Map<Long, Integer> currentStaticInitTask;
```

Listing 4.14: Depth counter map for nested static initialization.

```java
if (currentStaticInitTask.containsKey(taskId)) {
    // Same task, nested initialization
    int count = currentStaticInitTask.get(taskId);
    count++;
    currentStaticInitTask.put(taskId, count);
} else {
    // New task trying to initialize
    currentStaticInitTask.put(taskId, 1);
}
```

Listing 4.15: Handling nested `START_STATIC_INIT_EVENT`.

4.7 Static Synchronized Methods

Static synchronized methods presented another challenge. These methods acquire the class-level lock (the `Class` object's monitor), which JMC must track to control scheduling decisions.

4.7.1 The Static Synchronized Problem

Listing 4.18 shows a test with a static synchronized method:

The `synchronized` keyword on a static method means the method acquires the lock on `TestClass.class` before executing. JMC needs to:

1. Detect that the method is static and synchronized
2. Register the class-level lock with the runtime
3. Instrument the method to interact with JMC's lock tracking

4.7.2 The NullPointerException Problem

When testing static synchronized methods, JMC encountered a `NullPointerException` during the unlock operation. The root cause was that `getLock(className.hashCode())` returned null because the lock was never registered for the class before the static synchronized method was called.

The issue arose when a class like `TestClass` doesn't have an explicit static initializer block. Without a `<clinit>` method, JMC had no opportunity to register the class-level lock during class initialization. When the static synchronized method was invoked, JMC attempted to acquire a lock that didn't exist in the lock registry, resulting in a null reference.

4.7.3 Initial Approach: Lock Registration in <clinit>

The initial approach attempted to register static synchronized locks during class initialization by inserting registration code into the `<clinit>` method, as seen in Listing 4.19.

```java
int count = currentStaticInitTask.get(taskId);
if (count == 1) {
    // Outermost initialization complete
    currentStaticInitTask.remove(taskId);
} else {
    // Still in nested initialization
    currentStaticInitTask.put(taskId, count - 1);
}
```

Listing 4.16: Handling `END_STATIC_INIT_EVENT` with depth decrement.

```java
if (currentStaticInitTask.isEmpty()) {
    return getActiveTasks();  // All tasks can proceed
} else {
    // Only initializing tasks
    return new HashSet<>(currentStaticInitTask.keySet());
}
```

Listing 4.17: Tracker returning active tasks based on initialization state.

However, this approach had a critical flaw: not all classes have static initializers. If a class has no static fields or static blocks, the compiler doesn't generate a `<clinit>` method. Creating one artificially just for lock registration adds complexity and can interfere with the JVM's class initialization semantics.

4.7.4 Solution: Lazy Lock Registration

The solution is to register the lock lazily, immediately before it's first used. In the `JmcSyncMethodVisitor`'s `addSyncMethod()` method, for static synchronized methods, the instrumentation now inserts a registration call before acquiring the lock, as seen in Listing 4.20.

This approach ensures that:

- No need to create or modify `<clinit>` methods
- The lock is registered on first access to any static synchronized method
- `registerSyncLock` is idempotent (only registers if not already registered)
- Works for classes with or without static initializers
- Multiple static synchronized methods in the same class all work correctly (first call registers, subsequent calls find the existing lock)

The key insight is that we call `registerSyncLock(className)` right before `syncMethodLock(className)` for static methods, ensuring the lock exists before we try to acquire it. The registration operation is idempotent, so calling it multiple times (if the class has multiple static synchronized methods) is safe and has no performance impact after the first call.

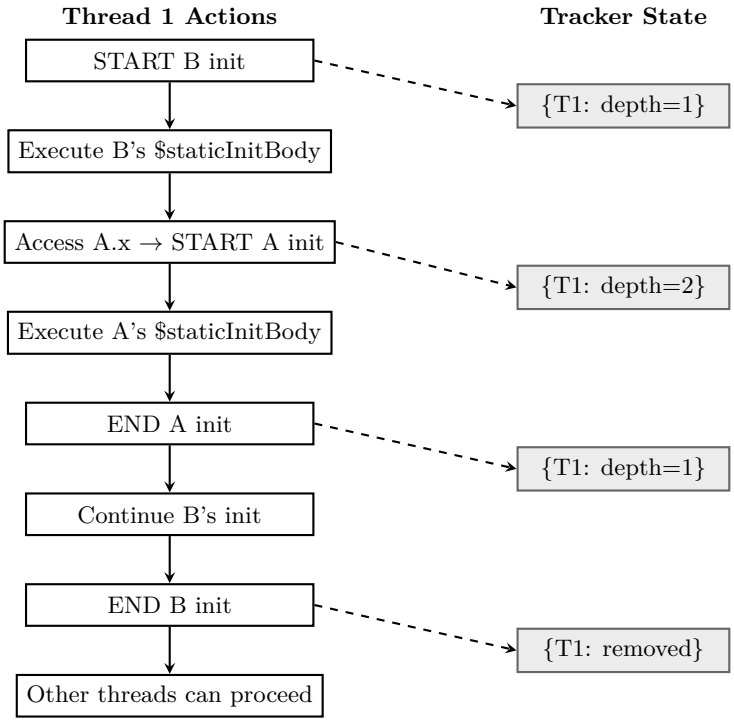

Figure 4.2: Nested static initialization with depth counter tracking.

4.8 Interface Static Fields

Java interfaces can have static fields, which are implicitly `public static final`. These require special handling because interfaces don't have traditional `<clinit>` methods in the same way classes do.

The `JmcStaticMethodVisitor` detects when it's visiting an interface and handles interface static fields specially:

1. Collect all interface static fields during the `visitField` phase
2. In `visitEnd`, create `$staticInitBody` that writes each field value
3. Create `$staticInitExplicit` and `$staticInitImplicit` methods
4. The interface's `<clinit>` is instrumented by `JmcStaticInitMethodVisitor` to register the interface

This ensures interface static fields can be reset across iterations just like class static fields, maintaining consistency in JMC's handling of static initialization.

4.9 Alternative Approaches Considered

Several alternative solutions were explored before arriving at the current approach. Each had significant limitations that made it impractical for JMC's requirements.

```java
// Test
String result = TestClass.getOrCreateValue();

// Class with static synchronized method
class TestClass {
    private static String value;

    private static synchronized String getOrCreateValue() {
        if (value == null) {
            value = "initialized";
        }
        return value;
    }
}
```

Listing 4.18: Static synchronized method test.

```java
static {
    // ... existing static initialization ...
    JmcRuntimeUtils.registerSyncLock(
        TestClass.class.getName());
}
```

Listing 4.19: Lock registration in `<clinit>` — initial approach.

4.9.1 Class Reloading via URLClassLoader

Another approach we tried was to reload classes for each iteration using a new `URLClassLoader`, thereby obtaining fresh static state. Since the JVM treats classes loaded by different class loaders as distinct types, this mechanism can isolate static state across iterations [32].

Classes loaded by different class loaders are treated as distinct types by the JVM, even if they originate from identical bytecode. As a result, objects created by one class loader cannot be safely cast to the same class loaded by another, leading to `ClassCastException` and violations of type compatibility [32].

Furthermore, class unloading depends on the reachability of the defining class loader. If any references to classes, instances, or associated metadata remain, the class loader cannot be garbage collected, preventing unloading and causing memory leaks [32].

Overall, these limitations introduce significant complexity and prevent the class loader-based approaches from providing reliable isolation for systematic testing.

4.9.2 Class Unloading

We further investigated whether we can rely on class unloading to reset static state between iterations. In the JVM, a class can only be unloaded when its defining class loader becomes unreachable and is garbage collected [32].

However, this approach is unreliable. Class unloading is not guaranteed to occur promptly, as it depends on garbage collection, which is inherently non-deterministic [32]. Additionally, ensuring that all references to a class and its loader are removed is difficult in practice, as hidden references may persist within the JVM or libraries.

```java
// try {
newMv.visitLabel(l0);

// For static methods, register the lock if not already registered
if (methodInfo.isStatic()) {
    // Check and register lock (idempotent operation)
    newMv.visitLdcInsn(className);
    newMv.visitMethodInsn(
        Opcodes.INVOKESTATIC,
        "org/mpi_sws/jmc/runtime/JmcRuntimeUtils",
        "registerSyncLock",
        "(Ljava/lang/String;)V",
        false
    );

    // Now lock
    newMv.visitLdcInsn(className);
} else {
    newMv.visitIntInsn(Opcodes.ALOAD, 0);
}

newMv.visitMethodInsn(
    Opcodes.INVOKESTATIC,
    "org/mpi_sws/jmc/runtime/JmcRuntimeUtils",
    "syncMethodLock",
    methodInfo.isStatic()
        ? "(Ljava/lang/String;)V"
        : "(Ljava/lang/Object;)V",
    false
);
```

Listing 4.20: Lazy lock registration before static synchronized method.

4.9.3 Reflection-Based Field Reset

A simpler approach was to use reflection to manually reset static field values to their initial state [18].

Java does not automatically rerun static initializers after a class is loaded. A common testing pattern is to provide a manual reset method, as seen in Listing 4.21.

```java
class Example {
    static int value = 42;

    public static void reset() {
        value = 42;  // Reset to initial value
    }
}
```

Listing 4.21: Manual reset pattern for static fields.

JMC could use reflection to discover all static fields and reset them to their initial values (0, null, false) or to values specified in annotations.

Reflection can reset field values, but it cannot re-execute static initialization blocks that

contain complex logic seen in Listing 4.22.

```
class Example {
    static List<String> items;
    static Map<String, Integer> cache;

    static {
        items = new ArrayList<>();
        items.add("default");

        cache = new HashMap<>();
        for (String item : items) {
            cache.put(item, item.length());
        }
    }
}
```

Listing 4.22: Complex static initializer with side effects.

Simply resetting `items` and `cache` to null doesn't recreate the initialization logic. The list wouldn't contain "default", and the cache wouldn't be populated.

By implementing this solution in JMC, we extend JMC's capacity further to be applicable to a wide variety of real-world programs.

Chapter 5

Lambdas

Lambda expressions are widely used in modern Java programming, enabling functional programming patterns and simplifying concurrent code through constructs such as streams, executors, and completable futures [9, 48, 51]. They are particularly prevalent in concurrent settings, where they are frequently used to express tasks and asynchronous computations. However, lambdas pose a challenge for JMC's instrumentation approach due to their implementation using the `invokedynamic` bytecode instruction [17].

5.0.1 The invokedynamic Instruction

The `invokedynamic` instruction enables dynamic method invocation by deferring method linkage to a bootstrap method that determines the call site's behavior at runtime [32]. Unlike traditional invocation instructions (`invokevirtual`, `invokestatic`, etc.), which have statically resolved targets, `invokedynamic` allows the target method to be computed dynamically.

An `invokedynamic` instruction is associated with a call site and consists of:

- **Name**: The method name (e.g., `"run"` for `Runnable`)
- **Method descriptor**: The type signature of the call site
- **Bootstrap method handle**: A reference to the method responsible for linking the call site
- **Bootstrap method arguments**: Additional static arguments passed to the bootstrap method

5.0.2 Bootstrap Method Handle

The bootstrap method handle specifies a method that is invoked during the first execution of the call site to establish its linkage [32]. This method returns a `CallSite` object, which encapsulates the target method to be invoked for subsequent executions.

For lambda expressions, the bootstrap method is typically `java.lang.invoke.LambdaMetafactory.metafactory` [35]. This method dynamically generates an implementation of the target functional interface, avoiding the need for separate class files and enabling efficient execution.

A `Handle` object describing the bootstrap method includes:

- **Tag**: The kind of method reference (e.g., static, virtual)
- **Owner**: The class defining the method
- **Name**: The method name
- **Descriptor**: The method signature
- **Interface flag**: Indicates whether the owner is an interface

5.0.3 Bootstrap Method Arguments

Bootstrap method arguments provide additional information used to configure the call site. These arguments are constants encoded in the class file and are specific to `invokedynamic` instructions [32].

For lambda expressions, these arguments typically include:

- **MethodType objects**: Representing the functional interface signature
- **Method handles**: Referencing the lambda body implementation
- **Additional type descriptors**: Captured variables or specialized types

These parameters allow the bootstrap method to construct an appropriate target method. For example, when a lambda captures an `AtomicInteger`, the bootstrap arguments encode both the captured type and the corresponding method handle, enabling the generation of a correctly typed implementation. The structure of an `invokedynamic` instruction is illustrated in Figure 5.1.

```
invokedynamic
|
|-- Call Site
|   |-- Name: "run"
|   |-- Descriptor: ()Ljava/lang/Runnable;
|
|-- Bootstrap Method
|   |-- Owner: java/lang/invoke/LambdaMetafactory
|   |-- Name: metafactory
|   |-- Descriptor: (...)Ljava/lang/invoke/CallSite;
|
|-- Bootstrap Arguments
    |-- MethodType: ()V              // SAM signature
    |-- MethodHandle: lambda$0(...)V // implementation
    |-- MethodType: ()V              // instantiated type
```

Figure 5.1: Structure of an `invokedynamic` instruction for lambda expressions.

5.1 The Instrumentation Challenge

JMC instruments Java programs by replacing concurrency primitives with JMC-controlled equivalents at the bytecode level. For example, `java.util.concurrent.atomic.AtomicInteger`

is replaced with `org.mpi_sws.jmc.api.util.concurrent.JmcAtomicInteger`. This transformation enables JMC to observe and control atomic operations during model checking.

However, lambda expressions introduce a subtle challenge. When a lambda captures or references such transformed types, the associated `invokedynamic` instruction may still encode type descriptors referring to the original Java types. Since `invokedynamic` defers linkage to runtime via bootstrap methods [32], inconsistencies between the rewritten program and the call site descriptors can lead to verification failures. Listing 5.1, and Listing 5.2 demonstrate how instrumentation captures the transformation of `AtomicInteger`.

```
AtomicInteger counter = new AtomicInteger(0);
ExecutorService executor = Executors.newFixedThreadPool(3);

for (int i = 0; i < 3; i++) {
    executor.submit(() -> {
        counter.incrementAndGet();
    });
}
```

Listing 5.1: Lambda capturing an `AtomicInteger` — before instrumentation.

```
JmcAtomicInteger counter = new JmcAtomicInteger(0);
JmcExecutorService executor =
    JmcExecutors.newFixedThreadPool(3);

for (int i = 0; i < 3; i++) {
    executor.submit(() -> {
        counter.incrementAndGet();
    });
}
```

Listing 5.2: After instrumentation — JMC types but stale lambda descriptor.

However, the corresponding `invokedynamic` instruction may still contain descriptors referring to `AtomicInteger`, as seen in Listing 5.3.

```
invokedynamic run()Ljava/lang/Runnable;
  bootstrap: LambdaMetafactory.metafactory(...)
  arguments: [
    Type: (Ljava/util/concurrent/atomic/AtomicInteger;)V
    Handle: lambda$test$0(AtomicInteger)
  ]
```

Listing 5.3: Stale `invokedynamic` descriptor referencing `AtomicInteger`.

At verification time, the JVM checks that operand stack types are compatible with the expected types encoded in the instruction [34]. In this case, the stack contains a `JmcAtomicInteger`, while the descriptor expects `AtomicInteger`. Since these types are not compatible, the verifier rejects the class with a `VerifyError` [32].

Without proper `invokedynamic` handling, JMC would produce verification errors as the one seen in Listing 8.15.

```
Caused by: java.lang.VerifyError: Bad type on operand stack
Exception Details:
  Location:
    org/apache/iceberg/jdbc/
    TestJdbcTableConcurrency.
    testConcurrentFastAppends$unsynchronized()V
    @256: invokedynamic
  Reason:
    Type 'org/mpisws/jmc/api/util/concurrent/
    JmcAtomicInteger' (current frame, stack[1])
    is not assignable to
    'java/util/concurrent/atomic/AtomicInteger'
  Current Frame:
    stack: { ...,
      'org/mpisws/jmc/api/util/concurrent/
       JmcAtomicInteger',
      integer,
      'org/apache/iceberg/Table', ... }
  Bytecode:
    ...
    @256: invokedynamic #105, 0
    ...
```

Listing 5.4: VerifyError from type mismatch in `invokedynamic`.

The error indicates a mismatch between the runtime operand stack and the expected types at the call site. While the bytecode offset identifies the problematic instruction, diagnosing such failures requires tracing the corresponding lambda expression and its captured variables.

5.2 Common Lambda Patterns

Lambda expressions appear in several common patterns in concurrent Java code, all of which require proper `invokedynamic` handling.

5.2.1 Executor Submissions

Listing 5.5 shows lambdas passed to `ExecutorService.submit()` or `execute()` often capture shared state [9]:

```java
AtomicInteger counter = new AtomicInteger(0);
ReentrantLock lock = new ReentrantLock();

executor.submit(() -> {
    lock.lock();
    try {
        counter.incrementAndGet();
    } finally {
        lock.unlock();
    }
});
```

Listing 5.5: Lambda capturing shared state with executor.

The lambda captures both `counter` and `lock`, so its descriptor must reference

`JmcAtomicInteger` and `JmcReentrantLock` after instrumentation.

5.2.2 CompletableFuture Chains

Listing 5.6 shows asynchronous pipelines using `CompletableFuture` frequently use lambdas [9].

```
CompletableFuture<Integer> future =
    CompletableFuture.supplyAsync(() -> {
        return atomicValue.get();
    });

future.thenApply(value -> {
    return value * 2;
}).thenAccept(result -> {
    resultCounter.set(result);
});
```

Listing 5.6: CompletableFuture chain with lambdas.

Each lambda in the chain may capture atomic variables or futures, requiring consistent type descriptor rewriting.

5.2.3 Thread Constructors

Creating threads with lambda-based `Runnable` implementations [51] are shown in Listing 5.7.

```
Thread thread = new Thread(() -> {
    atomicFlag.set(true);
});
thread.start();
```

Listing 5.7: Thread creation with lambda-based Runnable.

The lambda's descriptor must match the JMC thread implementation.

5.2.4 Parallel Streams

Parallel streams may use lambdas that access shared atomic counters [48], as seen in Listing 5.8.

```
list.parallelStream().forEach(item -> {
    counter.incrementAndGet();
    process(item);
});
```

Listing 5.8: Parallel stream with lambda accessing shared counter.

5.3 Solution: Type Descriptor Rewriting

The solution involves intercepting `invokedynamic` instructions during bytecode instrumentation and rewriting their type descriptors to replace Java concurrency types with JMC equivalents.

This must be done consistently across all components of the instruction: the method descriptor, the bootstrap method handle, and the bootstrap method arguments.

5.3.1 Visitor Implementation

Each visitor that handles type replacement (`JmcAtomicVisitor`, `JmcThreadVisitor`, `JmcReentrantLockVisitor`, `JmcFutureVisitor`) overrides the `visitInvokeDynamicInsn` method to perform this rewriting.

The method signature is shown in Listing 5.9.

```
public void visitInvokeDynamicInsn(
    String name,
    String descriptor,
    Handle bootstrapMethodHandle,
    Object... bootstrapMethodArguments)
```

Listing 5.9: `visitInvokeDynamicInsn` method signature.

5.3.2 Rewriting Strategy

The rewriting process follows these steps:

1. Detect Relevant Instructions. Check if the `invokedynamic` instruction involves types that JMC replaces. For `JmcAtomicVisitor`, this means checking if the descriptor or bootstrap method handle references `java.util.concurrent.atomic.Atomic` types as seen in Listing 5.10.

```
boolean isAtomicType =
    descriptor.contains(
        "java/util/concurrent/atomic/Atomic")
    || (bsm != null && bsm.getOwner().contains(
        "java/util/concurrent/atomic/Atomic"));
```

Listing 5.10: Detecting relevant `invokedynamic` instructions.

2. Rewrite Method Descriptor. Replace Java types with JMC types in the instruction's method descriptor, as shown in Listing 5.11.

```
String newDescriptor = replaceDescriptor(descriptor);
```

Listing 5.11: Rewriting the method descriptor.

The `replaceDescriptor` method uses string replacement to substitute type names shown in Listing 5.12.

3. Rewrite Bootstrap Method Handle. If the bootstrap method handle itself references replaced types, create a new handle with updated types as shown in Listing 5.13.

```java
private static String replaceDescriptor(String desc) {
    return desc
      .replace(
        "java/util/concurrent/atomic/AtomicInteger",
        "org/mpi_sws/jmc/api/util/concurrent/"
            + "JmcAtomicInteger")
      .replace(
        "java/util/concurrent/atomic/AtomicBoolean",
        "org/mpi_sws/jmc/api/util/concurrent/"
            + "JmcAtomicBoolean")
      // ... other atomic types
}
```

Listing 5.12: Type replacement in descriptors.

```java
Handle newBsm = bsm;
if (bsm != null) {
    String newOwner = replaceType(bsm.getOwner());
    String newBsmDesc =
        replaceDescriptor(bsm.getDesc());
    newBsm = new Handle(
        bsm.getTag(),
        newOwner,
        bsm.getName(),
        newBsmDesc,
        bsm.isInterface()
    );
}
```

Listing 5.13: Rewriting the bootstrap method handle.

4. Rewrite Bootstrap Method Arguments. Process each bootstrap argument, rewriting type descriptors in `Type` objects and method handles as shown in Listing 5.14.

5. Emit Modified Instruction. Call the superclass method with the rewritten components like Listing 5.15.

Handling Different Argument Types

Bootstrap method arguments may take several forms, each requiring specific handling during instrumentation.

Type objects represent classes and interfaces. If a `Type` refers to a replaced class (e.g., `AtomicInteger`), it must be rewritten to the corresponding JMC type (e.g., `JmcAtomicInteger`). This is achieved by constructing a new `Type` instance with the updated descriptor.

Method handles reference methods, including the lambda body. If a method handle's descriptor contains replaced types, both the descriptor and, if necessary, the owner class must be rewritten. A new `Handle` object is then created with the updated components.

Primitive values and strings are preserved unchanged, as they do not encode type-level information.

By correctly rewriting all bootstrap method arguments, JMC ensures consistency between

```java
Object[] newBsmArgs = new Object[bsmArgs.length];
for (int i = 0; i < bsmArgs.length; i++) {
    if (bsmArgs[i] instanceof Type t) {
        String className = t.getInternalName();
        newBsmArgs[i] =
            Type.getType(replaceType(className));
    }
    if (bsmArgs[i] instanceof Handle h) {
        String desc = replaceDescriptor(h.getDesc());
        newBsmArgs[i] = new Handle(
            h.getTag(),
            replaceType(h.getOwner()),
            h.getName(),
            desc,
            h.isInterface()
        );
    }
}
```

Listing 5.14: Rewriting bootstrap method arguments.

```java
super.visitInvokeDynamicInsn(
    name, newDescriptor, newBsm, newBsmArgs);
```

Listing 5.15: Emitting the modified `invokedynamic` instruction.

instrumented bytecode and dynamically linked call sites. This resolves the `invokedynamic` mismatch and enables JMC to support modern Java features such as lambda expressions, thereby extending its applicability to real-world concurrent applications.

Chapter 6

Executors and Futures

Java's executor framework provides high-level abstractions for concurrent task execution, allowing developers to focus on task logic rather than thread management details [9]. The `java.util.concurrent` package includes `ExecutorService` for managing thread pools, `ScheduledExecutorService` for time-based task scheduling, and various `Future` implementations for representing asynchronous computation results.

Supporting executors and futures in JMC required significant extensions beyond the initial implementation. While JMC already had custom implementations (`JmcExecutorService`, `JmcFuture`, `JmcFutureTask`) and corresponding visitors, these components were incomplete and not integrated into the visitor chain. Our solution involved fixing type mappings, extending the class hierarchy, implementing scheduled executor support, and solving resource management challenges.

6.1 Executors and Futures in Java

The executor framework provides a flexible infrastructure for managing concurrent task execution [9].

6.1.1 ExecutorService and Thread Pools

`ExecutorService` is the core interface for managing a pool of worker threads. Instead of creating threads directly, applications submit tasks (as `Runnable` or `Callable` objects) to an executor, which manages their execution [9] as seen in Listing 6.1.

```
ExecutorService executor =
    Executors.newFixedThreadPool(4);

executor.submit(() -> {
    // Task logic
    processData();
});

executor.shutdown();
```

Listing 6.1: Basic executor usage with task submission.

The `Executors` utility class provides factory methods for common configurations [9]:

- `newFixedThreadPool(int)`: Creates a thread pool with a fixed number of threads
- `newSingleThreadExecutor()`: Creates an executor with a single worker thread
- `newCachedThreadPool()`: Creates a thread pool that creates new threads as needed
- `newScheduledThreadPool(int)`: Creates a thread pool for scheduled task execution

The most common implementation is `ThreadPoolExecutor` [9], which provides fine-grained control over thread pool behaviour, including core pool size, maximum pool size, keep-alive time, and work queue configuration.

6.1.2 Future and Asynchronous Results

`Future` represents the result of an asynchronous computation. When a task is submitted to an executor, it returns a `Future` that can be used to retrieve the result, check completion status, or cancel the task [9] as Listing 6.2 shows.

```
Future<Integer> future = executor.submit(() -> {
    return computeValue();
});

// Blocks until result is available
Integer result = future.get();
```

Listing 6.2: Future for asynchronous result retrieval.

`FutureTask` is a concrete implementation that wraps a `Callable` or `Runnable` and provides `Future` semantics. It can be submitted to an executor or run directly.

6.1.3 ScheduledExecutorService

`ScheduledExecutorService` extends `ExecutorService` with methods for delayed and periodic task execution [9] as seen in Listing 6.3.

```
ScheduledExecutorService scheduler =
    Executors.newScheduledThreadPool(2);

// Execute after 5 seconds
scheduler.schedule(() -> {
    sendNotification();
}, 5, TimeUnit.SECONDS);

// Execute every 10 seconds
scheduler.scheduleAtFixedRate(() -> {
    checkHealth();
}, 0, 10, TimeUnit.SECONDS);
```

Listing 6.3: ScheduledExecutorService for delayed and periodic tasks.

`ScheduledFuture` extends `Future` with methods to query the remaining delay before execution.

6.2 The Instrumentation Challenge

The precise challenges included ensuring that the custom types defined in JMC were mapped to their corresponding Java counterparts.

6.2.1 Initial State: Incomplete Implementation

JMC already had custom implementations for executors and futures:

- `JmcExecutorService`: A JMC-controlled executor service implementation
- `JmcFuture`: A JMC-controlled future implementation
- `JmcFutureTask`: A JMC-controlled future task implementation
- `JmcExecutors`: Factory methods for creating JMC executors
- `JmcFutureVisitor`: Bytecode visitor for instrumenting executor and future usage

However, these components had several limitations. `JmcFutureVisitor` was commented out in the visitor chain, so executor instrumentation was not applied. Type mappings in `JmcExecutors` methods were incorrect, causing type mismatches. `JmcExecutorService` implemented the `ExecutorService` interface but did not extend `ThreadPoolExecutor`, limiting compatibility. No support for `ScheduledExecutorService` and `ScheduledFuture` was found. There was no mechanism to ensure executors were properly shut down between iterations.

6.2.2 Type Mapping Challenges

Real-world applications often use executors in patterns that require specific type relationships. Listing 6.4 shows a pattern from Apache Iceberg [3].

```
this.executorService =
    MoreExecutors.getExitingExecutorService(
        (ThreadPoolExecutor)
            Executors.newFixedThreadPool(poolSize)
    );
```

Listing 6.4: Apache Iceberg executor pattern with ThreadPoolExecutor cast.

This code:

1. Creates a fixed thread pool using `Executors.newFixedThreadPool()`
2. Casts the result to `ThreadPoolExecutor` (the concrete implementation)
3. Wraps it with Guava's `MoreExecutors.getExitingExecutorService()`, which expects a `ThreadPoolExecutor`

For JMC to handle this pattern, `JmcExecutors.newFixedThreadPool()` must return an instance that can be cast to `ThreadPoolExecutor`, provides JMC-controlled scheduling and maintains compatibility with third-party libraries.

The initial implementation, where `JmcExecutorService` only implemented the `ExecutorService` interface, could not satisfy these requirements.

6.2.3 Interface vs. Concrete Implementation

Another challenge was deciding which types to instrument. Java's executor framework has both interfaces (`ExecutorService`, `Future`) and concrete implementations (`ThreadPoolExecutor`, `FutureTask`).

Instrumenting interfaces would require replacing every implementation, including third-party and application-specific implementations. This approach is impractical and error-prone.

The solution is to instrument only concrete implementations from `java.util.concurrent`, allowing JMC to control the standard executor implementations while leaving custom implementations untouched.

6.3 Common Executor Patterns

Real-world applications use executors in various patterns, all of which JMC must support.

6.3.1 Factory Method Usage

The most common pattern uses `Executors` factory methods as seen in Listing 6.5.

```
ExecutorService executor =
    Executors.newFixedThreadPool(poolSize);
Future<Result> future =
    executor.submit(() -> computeResult());
Result result = future.get();
executor.shutdown();
```

Listing 6.5: Factory method usage pattern.

6.3.2 Third-Party Library Integration

Applications often use third-party libraries that wrap or enhance executors. Guava's `MoreExecutors` is particularly common as shown in Listing 6.6.

```
ThreadPoolExecutor pool =
    (ThreadPoolExecutor)
        Executors.newFixedThreadPool(4);
ExecutorService exitingExecutor =
    MoreExecutors.getExitingExecutorService(pool);
```

Listing 6.6: Guava's MoreExecutors wrapping a ThreadPoolExecutor.

6.3.3 Scheduled Task Execution

Time-based scheduling is common in server applications as seen in Listing 6.7.

```
ScheduledExecutorService scheduler =
    Executors.newScheduledThreadPool(2);

scheduler.scheduleAtFixedRate(() -> {
    performHealthCheck();
}, 0, 30, TimeUnit.SECONDS);
```

Listing 6.7: Scheduled task execution pattern.

6.3.4 Internal Wrapper Classes

The `Executors` class uses internal wrapper classes like `DelegatedExecutorService` and `FinalizableDelegatedExecutorService` to limit the exposed API. These wrappers must also be handled by JMC.

6.4 Solution: Hierarchical Type Mapping and Lifecycle Management

The solution involved multiple coordinated changes: enabling the visitor chain, fixing type mappings, restructuring the class hierarchy, implementing scheduled executor support, and adding resource management.

6.4.1 Step 1: Enabling the Visitor Chain

The first step was to uncomment `JmcFutureVisitor` in the visitor chain. This visitor was already implemented but disabled, preventing any executor instrumentation from occurring.

Adding it to the chain immediately revealed the type mapping issues that needed to be addressed.

6.4.2 Step 2: Fixing JmcExecutors Type Mappings

The `JmcExecutors` factory methods had incorrect return types and method signatures. The methods needed to return instances compatible with `ThreadPoolExecutor` to support patterns like the Guava example.

The corrected mappings ensure that:

- `JmcExecutors.newFixedThreadPool()` returns a `JmcExecutorService` instance
- `JmcExecutors.newSingleThreadExecutor()` returns a `JmcExecutorService` instance
- All factory methods support both parameterless and `ThreadFactory`-accepting variants

6.4.3 Step 3: Restructuring JmcExecutorService

Initially, `JmcExecutorService` was designed to extend `ExecutorService`.

However, this design failed for code that casts executors to `ThreadPoolExecutor` or checks `instanceof ThreadPoolExecutor`. The solution was to make `JmcExecutorService` extend `ThreadPoolExecutor`.

This change required:

1. Calling the appropriate `ThreadPoolExecutor` constructor
2. Overriding methods to integrate with JMC's scheduling
3. Maintaining the existing `JmcWorkerPool` functionality
4. Ensuring compatibility with `ThreadPoolExecutor`-specific methods

The restructured `JmcExecutorService` now extends `ThreadPoolExecutor`, making it compatible with code that expects the concrete class while providing JMC-controlled scheduling.

6.4.4 Step 4: Implementing Scheduled Executor Support

Many applications use `ScheduledExecutorService` [9] for time-based task execution. JMC needed custom implementations to handle scheduling without relying on real time.

JmcScheduledExecutorService. This class provides JMC-controlled scheduled execution:

- Extends `JmcExecutorService` to inherit thread pool functionality
- Implements `ScheduledExecutorService` interface methods
- Replaces time-based delays with logical time controlled by the scheduler
- Tracks scheduled tasks and makes them eligible for execution based on logical time advancement

JmcScheduledFuture. This class represents the result of a scheduled task:

- Extends `JmcFuture` to inherit future functionality
- Implements `ScheduledFuture` interface methods
- Provides delay tracking based on logical time
- Supports cancellation with proper state transitions

JMC does not model real-time delays. The implementation provides workarounds for delay-related methods.

Java Type		JMC Type
`Executors`	$\longrightarrow$	`JmcExecutors`
`ThreadPoolExecutor`	$\longrightarrow$	`JmcExecutorService`
`DelegatedExecutorService`	$\longrightarrow$	`JmcExecutorService`
`FinalizableDelegatedExecutorService`	$\longrightarrow$	`JmcExecutorService`
`CompletableFuture`	$\longrightarrow$	`JmcCompletableFuture`

Note: `ExecutorService` and `Future` interfaces are not mapped. Only concrete implementations are instrumented.

Figure 6.1: Type mappings for executors and futures in JMC.

6.4.5 Step 5: Visitor Changes for Type Mapping

The `JmcFutureVisitor` needed updates to handle the new type mappings. Figure 6.1 illustrates the complete type mapping strategy.

The visitor implements these mappings by:

1. Detecting method calls to `Executors` factory methods
2. Replacing the owner class from `java.util.concurrent.Executors` to `org.mpi_sws.jmc.api.util.concurrent.JmcExecutors`
3. Updating field descriptors that reference `ThreadPoolExecutor` to reference `JmcExecutorService`
4. Handling internal wrapper classes by mapping them to `JmcExecutorService`
5. Rewriting `invokedynamic` instructions that capture executor or future types (as described in the Lambdas section)

6.4.6 Step 6: Resource Management with TrackExecutors

After implementing all the type mappings and custom implementations, a critical problem emerged: running tests with executors for many iterations caused out-of-memory errors with the message "unable to create native thread."

The root cause was that executors create native OS threads, and if executors are not explicitly shut down, these threads remain alive [9]. Over thousands of iterations, the accumulated threads exhaust system resources.

Consider a test that creates an executor but doesn't shut it down as seen in Listing 6.8.

When JMC runs this test for 1000 iterations, it creates 4000 threads (4 per iteration) that are never cleaned up. Eventually, the OS refuses to create more threads, causing the test to fail with an OOM error.

TrackExecutors. Similar to `TrackStaticInit`, we implemented `TrackExecutors`, a tracker that monitors executor lifecycle and forces shutdown between iterations.

The tracker maintains a registry of all created executors:

```java
@Test
public void testConcurrentOperations() {
    ExecutorService executor =
        Executors.newFixedThreadPool(4);

    for (int i = 0; i < 100; i++) {
        executor.submit(() -> processItem(i));
    }

    // Missing: executor.shutdown()
}
```

Listing 6.8: Test with missing executor shutdown — causes thread leak.

```java
private final Set<ExecutorService> registeredExecutors;
```

Listing 6.9: TrackExecutors registry.

When an executor is created, it fires an `EXECUTOR_SHUTDOWN_EVENT` with action "register" as Listing 6.10 demonstrates.

```java
JmcRuntimeUtils.executorShutdownEvent(
    "register", this);
```

Listing 6.10: Executor registration event.

The tracker's `updateEvent` method handles this event as seen in Listing 6.11.

```java
if (type == JmcRuntimeEvent.Type
        .EXECUTOR_SHUTDOWN_EVENT) {
    String action = event.getParam("action");
    ExecutorService executor =
        event.getParam("executor");
    if ("register".equals(action)) {
        registeredExecutors.add(executor);
    }
}
```

Listing 6.11: TrackExecutors event handling.

At the end of each iteration, the tracker's `reset` method calls `shutdownExecutors` as seen in Listing 6.12.

This ensures that all executors are properly shut down between iterations, preventing thread accumulation and OOM errors.

A new runtime event type was added to support this functionality, as seen in Listing 6.13.

This event carries the executor instance and the action (register/shutdown), allowing the tracker to manage the executor lifecycle.

```java
public void shutdownExecutors() {
    for (ExecutorService executor
            : registeredExecutors) {
        if (executor instanceof
                JmcExecutorService jmcExecutor) {
            if (!jmcExecutor.isShutdown()) {
                jmcExecutor.shutdown();
            }
        }
        if (executor instanceof
                JmcScheduledExecutorService jmcExec) {
            if (!jmcExec.isShutdown()) {
                jmcExec.shutdown();
            }
        }
    }
}
```

Listing 6.12: Forced executor shutdown between iterations.

```java
public enum Type {
    // ... existing events
    EXECUTOR_SHUTDOWN_EVENT
}
```

Listing 6.13: EXECUTOR_SHUTDOWN_EVENT runtime event type.

Chapter 7

Object Class

The `java.lang.Object` class is the root of Java's class hierarchy [21]. Every class in Java implicitly extends `Object`, inheriting its methods and semantics [21, 41]. The support `of` object class methods in JMC is crucial because these methods are ubiquitous in Java programs, appearing in synchronization primitives [49], collection implementations, and application logic. However, instrumenting these methods is challenging due to their native implementations [40] and the fact that they are tightly integrated with JVM execution.

The `Object` class provides two categories of methods that require JMC support: synchronization primitives [49, 21] (`wait()`, `notify()`, `notifyAll()`) and native utility methods [21] (`hashCode()`, `toString()`, `equals()`, `clone()`, `finalize()`). Each category presents different instrumentation challenges.

7.1 Object Class in Java

The `Object` class defines fundamental methods that all Java objects inherit.

7.1.1 Synchronization Methods

Java provides built-in monitor-based synchronization through the `Object` class [21] as seen in Listing 7.1.

```
synchronized (object) {
    while (!condition) {
        object.wait();   // Release lock and wait
    }
    // Perform work
    object.notifyAll();  // Wake up waiting threads
}
```

Listing 7.1: Monitor-based synchronization with wait/notify.

wait(): Causes the current thread to release the monitor lock on the object and wait until another thread calls `notify()` or `notifyAll()` on the same object. The thread must own the object's monitor (be inside a `synchronized` block on that object) or an `IllegalMonitorStateException` is thrown [21].

notify(): Wakes up a single thread waiting on the object's monitor. The choice of which thread to wake is arbitrary [21].

notifyAll(): Wakes up all threads waiting on the object's monitor [21].

7.1.2 Native Utility Methods

The `Object` class provides several utility methods, many of which are native:

hashCode(): Returns an integer hash code for the object. The default implementation (in native code) typically returns a value derived from the object's memory address. Classes often override this method to provide value-based hashing. The method is declared as `public native int hashCode()` [21].

toString(): Returns a string representation of the object. The default implementation returns `ClassName@HashCode`. The method calls `hashCode()` internally [21].

equals(Object): Compares two objects for equality. The default implementation uses reference equality (`==`). Classes override this method to provide value-based equality [21].

clone(): Creates a shallow copy of the object. The method is `protected` and requires the class to implement `Cloneable` [21].

finalize(): Called by the garbage collector before reclaiming the object's memory. Deprecated in Java 9 but still present in legacy code [21].

Many `Object` class methods are annotated with `@IntrinsicCandidate`, indicating that the JVM's JIT compiler may replace them with hand-crafted assembly code or compiler intrinsics for performance. Methods like `hashCode()`, `getClass()`, and `notify()` are intrinsic candidates.

When a method is compiled as an intrinsic, the JIT compiler may bypass the method's bytecode entirely, executing optimized native code instead [16].

7.2 The Runtime Challenge

Instrumenting `Object` class methods presents several interconnected challenges.

7.2.1 Cannot Replace Object Class

Unlike `AtomicInteger` or `ExecutorService`, JMC cannot create a `JmcObject` class and replace `Object` with it. Every class in Java extends `Object`, so replacing it would require rewriting the entire class hierarchy, including JDK classes, third-party libraries, and application code. This approach is infeasible.

Instead, JMC must intercept calls to `Object` methods and redirect them to JMC-controlled implementations without modifying the `Object` class itself.

7.2.2 Native Method Instrumentation

Many methods in `java.lang.Object` are annotated with `@IntrinsicCandidate`, indicating that they may be replaced by JVM compiler intrinsics during execution [16, 13]. Methods such as `hashCode()`, `getClass()`, and `notify()` are typical examples.

For example, consider a class that overrides `hashCode()` as seen in Listing 7.2.

```java
class Item {
    private int value;

    @Override
    public int hashCode() {
        return Objects.hashCode(value);
    }
}
```

Listing 7.2: Class overriding `hashCode()` — JIT may still intrinsify.

Even though `Item.hashCode()` has bytecode; the JIT compiler may inline and optimize it aggressively, potentially bypassing instrumented field accesses within the method.

7.3 Motivating Example: HashSet with Concurrent Access

To illustrate the problem, consider a test that uses `HashSet` with custom objects in Listing 7.3.

A concurrent test over this class is shown in Listing 7.4.

Without proper instrumentation of `hashCode()`, this test may hang. When `hashCode()` is intrinsified, its execution may not follow the instrumented bytecode path, preventing JMC from introducing scheduling points [16, 13].

7.4 Attempted Solutions

We implemented several approaches before arriving at the current solution.

7.4.1 Initial Decision: No Instrumentation

Initially, JMC did not instrument methods such as `hashCode()`, `toString()`, or `equals()`. However, the motivating example demonstrated that when `hashCode()` is invoked within concurrent operations (e.g., during `HashSet` lookups), it may be intrinsified by the JVM, causing execution to deviate from the instrumented bytecode path. As a result, threads may run without turning on the JMC scheduler [16, 13].

7.4.2 Attempted Solution 1: Clearing Access Flags

We then tried to clear the `ACC_NATIVE` and `ACC_FINAL` flags on methods, thinking that it might allow instrumentation. Listing 7.5 shows this.

This approach failed because overridden methods do not have these flags. When a class overrides `hashCode()`, the overriding method is a normal Java method without `ACC_NATIVE` or `ACC_FINAL` flags.

7.4.3 Attempted Solution 2: JVM Flags to Disable Intrinsics

We tried to disable intrinsic optimizations using these JVM flags as seen in Listing 7.6.

```java
static class Item {
    private int value;

    public Item(int value) {
        this.value = value;
    }

    @Override
    public int hashCode() {
        return Objects.hashCode(value);
    }

    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Item)) return false;
        Item other = (Item) obj;
        return this.value == other.value;
    }
}

static class Container {
    private Set<Item> items;

    public Container() {
        this.items = new HashSet<>();
    }

    public void addItem(Item item) {
        this.items.add(item);
    }

    public int computeHash() {
        return items.stream()
            .mapToInt(Item::hashCode).sum();
    }
}
```

Listing 7.3: Item and Container classes with hashCode/equals.

We believed that these flags were intended to limit the use of compiler intrinsics, forcing execution to follow the instrumented bytecode paths. However, this approach proved ineffective as we did not see any difference in using these flags.

7.4.4 Attempted Solution 3: Renaming Overridden Methods

We then tried renaming overridden methods:

- If a class overrides `hashCode()`, rename it to `hashCode__jmc__`
- If a class does not override `hashCode()`, create `hashCode__jmc__` that calls `super.hashCode()`
- Intercept calls to `hashCode()` and redirect them to `hashCode__jmc__`

This approach worked for simple cases but failed in real-world scenarios. Consider a class hierarchy depicted in Listing 7.7.

```java
@JmcCheck
@JmcCheckConfiguration(
    numIterations = 1, debug = true)
public void testHashCode()
        throws InterruptedException {
    Container container = new Container();
    ...

    Thread t1 = new Thread(() -> {
        int hash = container.computeHash();
        if (hash < 0) {
            System.out.println("Unexpected");
        }
    });

    Thread t2 = new Thread(() -> {
        int hash = container.computeHash();
        ...
    });

    t1.start();
    t2.start();
    t1.join();
    t2.join();
}
```

Listing 7.4: Concurrent test using hashCode — hangs without instrumentation.

After renaming, `Namespace.hashCode()` becomes `Namespace.hashCode__jmc__`. Now consider code that uses `Namespace` as a key in a `ConcurrentHashMap` as seen in Listing 7.8.

When `ConcurrentHashMap` calls `namespace.hashCode()`, the JVM cannot find the `hashCode()` method in `Namespace` (it was renamed). The JVM then searches up the class hierarchy, eventually finding `Object.hashCode()`.

However, `Object.hashCode()` is native and uninstrumented. The call executes without JMC control, causing the same hang problem.

To fix this, JMC would need to provide custom implementations of `ConcurrentHashMap`, `HashMap`, `HashSet`, and every other collection that uses `hashCode()`. This approach is unrealistic; it would require reimplementing large portions of the JDK.

JMC's runtime operates alongside the JVM runtime, which means that parts of the program execute through JMC's instrumented code, while other parts (like JDK collection internals) execute through the JVM's native code. Renaming methods breaks the boundary between these two execution modes.

7.5 Current Solution: Delegation with Reflection

The working solution avoids renaming original methods and instead adds extra methods that can be invoked via reflection [18].

```java
@Override
public MethodVisitor visitMethod(
        int access, String name, String desc,
        String signature, String[] exceptions) {
    if ("hashCode".equals(name)
            && "()I".equals(desc)) {
        // Remove native, abstract, and final flags
        access &= ~(Opcodes.ACC_NATIVE
            | Opcodes.ACC_ABSTRACT
            | Opcodes.ACC_FINAL);
    }
    return super.visitMethod(
        access, name, desc, signature, exceptions);
}
```

Listing 7.5: Attempted fix: clearing access flags on native methods.

```
-XX:+UnlockDiagnosticVMOptions
-XX:+InlineIntrinsics=false
-XX:+PrintIntrinsics
```

Listing 7.6: JVM flags to disable intrinsics — ineffective.

7.5.1 Part 1: Wait/Notify/NotifyAll

For synchronization methods, JMC uses method interception to redirect calls to static methods in the `JmcObject` utility class.

JmcWaitNotifyVisitor. This visitor intercepts calls to `wait()`, `notify()`, and `notifyAll()` as seen in Listing 7.9.

The visitor replaces `INVOKEVIRTUAL` instructions targeting `Object` methods with `INVOKESTATIC` instructions targeting `JmcObject` static methods.

JmcObject Implementation. The `JmcObject.objectWait()` method integrates with JMC's scheduling. Listing 7.10 demonstrates this.

The implementation ensures that `wait()` releases the lock, yields to the scheduler, and reacquires the lock upon wakeup, matching Java's semantics while providing JMC control.

TrackWaitNotify. This tracker extends `TrackLocks` to manage wait/notify state:

- `waitingTasks`: Maps object IDs to sets of tasks waiting on that object
- `availableTasks`: Maps object IDs to sets of tasks that have been notified but not yet woken

When a task calls `wait()`, the tracker removes it from active tasks and adds it to `waitingTasks`. When `notify()` is called, the tracker moves tasks from `waitingTasks` to `availableTasks`. When a task wakes up, it moves from `availableTasks` back to active tasks.

This mechanism ensures that only one task wakes up per `notify()` call, and all tasks wake up for `notifyAll()`, matching Java's semantics.

```java
class Namespace {
    private String[] levels;

    @Override
    public int hashCode() {
        return Arrays.hashCode(levels);
    }
}
```

Listing 7.7: Namespace class with overridden hashCode.

```java
ConcurrentHashMap<Namespace, Value> map =
    new ConcurrentHashMap<>();
map.put(namespace, value);
```

Listing 7.8: ConcurrentHashMap calling hashCode on renamed method — fails.

7.5.2 Part 2: Native Utility Methods

For `hashCode()`, `toString()`, `equals()`, and `finalize()`, JMC uses a delegation pattern that preserves original methods while adding instrumented alternatives.

The visitor, as seen in Listing 7.11, adds extra methods without modifying the originals:

1. Keep `hashCode()`, `toString()`, `equals()` unchanged
2. Add `jmcHashCode()`, `jmcToString()`, `jmcEquals(Object)` methods
3. These `jmcXxx()` methods call `super.xxx()`
4. For `finalize()`, rename overridden versions to `jmcFinalize()` to prevent them from being called

```java
// Original code
object.wait();

// After instrumentation
JmcObject.objectWait(object);
```

Listing 7.9: Wait/notify instrumentation — before and after.

```java
public static void objectWait(Object o)
        throws InterruptedException {
    JmcReentrantLock lock =
        JmcRuntimeUtils.getSyncLock(o);
    ...

    // Fire WAIT_EVENT
    JmcRuntimeEvent event =
        new JmcRuntimeEvent.Builder()
            .type(JmcRuntimeEvent.Type.WAIT_EVENT)
            .taskId(JmcRuntime.currentTask())
            .param("object", o)
            .build();
    JmcRuntime.updateEventAndYield(event);

    // Reacquire lock after wakeup
    lock.lock();

    // Fire WAKEUP_EVENT
    event = new JmcRuntimeEvent.Builder()
        .type(JmcRuntimeEvent.Type.WAKEUP_EVENT)
        .taskId(JmcRuntime.currentTask())
        .param("object", o)
        .build();
    JmcRuntime.updateEventAndYield(event);
}
```

Listing 7.10: JmcObject.objectWait() implementation.

The `JmcObject` class provides handler methods that use reflection to invoke `jmcXxx()` methods as seen in Listing 7.12.

This approach works because:

- Original methods remain unchanged, so JVM code (like `ConcurrentHashMap`) can call them normally
- `jmcXxx()` methods provide instrumented alternatives that call `super.xxx()`
- Reflection allows dynamic dispatch to `jmcXxx()` methods when needed
- The JVM's method resolution finds the original methods, preventing the hierarchy traversal problem

The `toString()` method required special care because `Object.toString()` internally calls `hashCode()`.

If `jmcToString()` calls `super.toString()`, it triggers `hashCode()`, which may be uninstrumented. To avoid this, `JmcObject` provides a custom `toString()` implementation as seen in Listing 7.13.

This implementation mimics `Object.toString()`'s behavior [21] while using `handleHashCode()` to ensure the hash code is computed through JMC's instrumented path.

7.5.3 Addressing the Infinite Read Problem

An additional issue emerged with `toString()` in `JmcRuntimeEvent`.

Listing 7.14 shows that the `params` field is a `Map`. When `toString()` is called on a `JmcRuntimeEvent`, it calls `params.toString()`, which iterates over the map entries and calls `toString()` on each value. If a value is an object with instrumented field accesses, this triggers read events, which create new `JmcRuntimeEvent` instances, which call `toString()`, creating an infinite loop.

The solution is a custom `paramToString()` method that carefully handles the map as seen in Listing 7.15.

For the "instance" parameter, the method uses `handleToString()`, which invokes `JmcObject.toString()` to avoid triggering instrumented field accesses. For other parameters, it uses the default `toString()`.

```java
@Override
public void visitEnd() {
    if (!isInterface) {
        // Always create jmcHashCode(), jmcEquals(),
        // jmcToString()
        createJmcMethod("hashCode",
            "()I", Opcodes.IRETURN);
        ...
    }
    super.visitEnd();
}

private void createJmcMethod(String methodName,
        String descriptor, int returnOpcode) {
    String jmcMethodName = "jmc"
        + Character.toUpperCase(methodName.charAt(0))
        + methodName.substring(1);

    ...

    if (mv != null) {
        ...

        // Special handling for toString
        if ("toString".equals(methodName)) {
            mv.visitVarInsn(Opcodes.ALOAD, 0);
            mv.visitMethodInsn(Opcodes.INVOKESTATIC,
                "org/mpi_sws/jmc/api/JmcObject",
                "toString",
                "(Ljava/lang/Object;)"
                    + "Ljava/lang/String;",
                false);
            ...
        } else {
            mv.visitVarInsn(Opcodes.ALOAD, 0);

            if ("equals".equals(methodName)) {
                mv.visitVarInsn(Opcodes.ALOAD, 1);
            }
            ...
            mv.visitMethodInsn(Opcodes.INVOKESPECIAL,
                superName, methodName,
                descriptor, false);
            mv.visitInsn(returnOpcode);

            ...
        }
        mv.visitEnd();
    }
}
```

Listing 7.11: JmcNativeMethodVisitor — creating jmcXxx() methods.

```java
public static int handleHashCode(Object obj) {
    if (obj == null) return 0;

    try {
        Method method =
            obj.getClass().getMethod("jmcHashCode");
        method.setAccessible(true);
        return (int) method.invoke(obj);
    } catch (NoSuchMethodException e) {
        return obj.hashCode();
    } catch (IllegalAccessException
            | InvocationTargetException e) {
        throw new RuntimeException(
            "Failed to invoke jmcHashCode", e);
    }
}
```

Listing 7.12: Reflection-based invocation of jmcHashCode().

```java
public static String toString(Object obj) {
    return obj.getClass().getName() + "@"
        + Integer.toHexString(
            handleHashCode(obj));
}
```

Listing 7.13: Custom JmcObject.toString() using handleHashCode().

```java
@Override
public String toString() {
    return "RuntimeEvent{" + "type=" + type
        + ", taskId=" + taskId
        + ", params=" + params + '}';
}
```

Listing 7.14: Original JmcRuntimeEvent.toString() — causes infinite loop.

```java
private String paramToString(
        Map<String, Object> params) {
    if (params == null) return "";
    StringBuilder sb = new StringBuilder();
    for (Map.Entry<String, Object> entry
            : params.entrySet()) {
        if (!sb.isEmpty()) {
            sb.append(", ");
        }
        Object o = entry.getValue();
        if (entry.getKey().equals("instance")) {
            sb.append(handleToString(o));
        } else {
            sb.append(o);
        }
    }
    return sb.toString();
}

@Override
public String toString() {
    return "RuntimeEvent{" + "type=" + type
        + ", taskId=" + taskId
        + ", params=" + paramToString(params) + '}';
}
```

Listing 7.15: Safe paramToString() avoiding infinite recursion.

Chapter 8

Miscellaneous

This chapter covers several smaller but important instrumentation challenges that arose during JMC's development. Each represents a specific edge case or integration issue that required careful handling to ensure correct model checking behaviour.

8.1 Enums

Java enums are special classes that extend `java.lang.Enum` and represent a fixed set of constants [8]. The JVM treats enums differently from regular classes, with special handling for their static initialization, serialization, and reflection behavior [8, 33].

8.1.1 The Challenge

Enums present unique instrumentation challenges because they combine static initialization (enum constants are static final fields) with complex compiler-generated code. The Java compiler generates synthetic methods like `values()` and `valueOf(String)` for every enum, along with a static initializer that creates the enum constants [33].

Consider the simple enum shown in Listing 8.1 and the corresponding bytecode generated by the compiler in Listing 8.2.

```
enum MyEnum {
    A, B, C;
}
```

Listing 8.1: Simple enum declaration.

Instrumenting this code is problematic. The enum constants are `static final` fields, which JMC's static initialization instrumentation would attempt to reset across iterations [8].

Moreover, the JVM enforces strict structural and type-safety constraints during class verification. Modifying enum bytecode can violate these constraints, potentially leading to verification errors or class loading failures [34].

```java
final class MyEnum extends Enum<MyEnum> {
    public static final MyEnum A;
    public static final MyEnum B;
    public static final MyEnum C;
    private static final MyEnum[] $VALUES;

    static {
        A = new MyEnum("A", 0);
        B = new MyEnum("B", 1);
        C = new MyEnum("C", 2);
        $VALUES = new MyEnum[] { A, B, C };
    }

    public static MyEnum[] values() {
        return $VALUES.clone();
    }

    public static MyEnum valueOf(String name) {
        return Enum.valueOf(MyEnum.class, name);
    }
}
```

Listing 8.2: Compiler-generated bytecode for enum.

8.1.2 Solution: Skip Enum Instrumentation

JMC's solution is to detect enum classes and skip instrumentation entirely. The `JmcIgnoreEnumVisitor` checks the class access flags for `ACC_ENUM` as seen in Listing 8.3.

In `JmcVisitor`, the main instrumentation entry point, enum detection happens before any other visitors are applied. This can be seen from Listing 8.4.

This approach leaves enum classes unmodified by JMC's instrumentation pipeline. The original bytecode is preserved, maintaining the expected enum semantics and JVM-defined behaviour.

8.2 Final Fields

The `final` keyword in Java specifies that a field cannot be reassigned after initialization. In addition to this immutability guarantee, the Java Memory Model provides special visibility guarantees for final fields, ensuring that their values are safely published once initialization completes [10].

8.2.1 The Challenge

JMC instruments field accesses to detect data races by tracking read and write operations. However, final fields introduce a subtle challenge: while they are immutable by definition and therefore do not participate in data races in the traditional sense, the JVM enforces strict constraints on how they are initialized, which complicates instrumentation.

An initial approach attempted to remove the `final` modifier during instrumentation, allowing JMC to reset field values across iterations, similar to the handling of static initialization.

```java
public class JmcIgnoreEnumVisitor
        extends ClassVisitor {
    private boolean isEnum;

    @Override
    public void visit(int version, int access,
            String name, String signature,
            String superName, String[] interfaces) {
        this.isEnum =
            (access & Opcodes.ACC_ENUM) != 0;
        super.visit(version, access, name,
            signature, superName, interfaces);
    }

    public boolean isEnum() {
        return isEnum;
    }
}
```

Listing 8.3: JmcIgnoreEnumVisitor detecting enum classes.

```java
JmcIgnoreEnumVisitor enumVisitor =
    new JmcIgnoreEnumVisitor(enumCw);
enumCr.accept(enumVisitor,
    ClassReader.SKIP_DEBUG
    | ClassReader.SKIP_FRAMES);

if (enumVisitor.isEnum()) {
    // Return original bytecode unchanged
    return classFileBuffer;
}
```

Listing 8.4: Early enum detection in JmcVisitor.

However, this approach did not work.

The JVM enforces that final fields may only be assigned during object construction (for instance fields) or within the class initializer `<clinit>` (for static fields) [10, 34]. Violating these constraints can result in verification errors, like Listing 8.5, at class loading time.

```
java.lang.VerifyError: Bad access to protected data
```

Listing 8.5: VerifyError from writing to final field.

Even when the `ACC_FINAL` flag is removed at the bytecode level, the JVM may still enforce constraints derived from the original class structure during verification [34]. These restrictions are designed to preserve immutability guarantees and prevent unsafe modification of fields that are intended to be constant.

8.2.2 Solution: Ignore Final Fields

The working solution is to skip the instrumentation of final fields entirely. The `JmcReadWriteVisitor` tracks final fields during the `visitField` phase and excludes them from read/write instrumen-

tation as seen in Listing 8.6.

```java
private final Set<String> finalFields =
    new HashSet<>();

@Override
public FieldVisitor visitField(int access,
        String name, String descriptor,
        String signature, Object value) {
    // Track final fields
    if ((access & Opcodes.ACC_FINAL) != 0) {
        finalFields.add(className + "/" + name);
    }
    return super.visitField(access, name,
        descriptor, signature, value);
}
```

Listing 8.6: Tracking final fields in JmcReadWriteVisitor.

When instrumenting field access instructions, the visitor checks if the field is final and skips instrumentation:

```java
@Override
public void visitFieldInsn(int opcode,
        String owner, String name,
        String descriptor) {
    // Skip final fields
    String fieldKey = owner + "/" + name;
    if (finalFields.contains(fieldKey)) {
        super.visitFieldInsn(opcode, owner,
            name, descriptor);
        return;
    }

    // Instrument non-final field access
    // ...
}
```

Listing 8.7: Skipping instrumentation for final fields.

This approach respects the JVM's constraints while avoiding unnecessary instrumentation overhead.

8.3 Synchronized Annotation Preservation

When JMC instruments synchronized methods, it removes the `synchronized` modifier and introduces explicit locking logic. During this transformation, method annotations must be preserved, including nested and complex annotation structures defined by the Java language [1].

8.3.1 The Challenge

Java annotations can use different value types, including primitives, strings, enums, arrays, and nested annotations [1]. When JMC creates a new method to replace a synchronized method, it must copy all annotations, keeping their full structure.

Consider a method with nested annotations as seen in Listing 8.8.

```
@JmcCheck
@JmcCheckConfiguration(
    numIterations = 10,
    strategy = @Strategy(
        type = StrategyType.RANDOM, seed = 42)
)
public synchronized void testMethod() {
    // ...
}
```

Listing 8.8: Method with nested annotations.

The `@JmcCheckConfiguration` annotation contains a nested `@Strategy` annotation. When JMC removes `synchronized` and creates a new method, it must recreate this entire annotation structure.

8.3.2 Solution: Recursive Annotation Copying

We use `AnnotationVisitor` from ASM [5] in `JmcSyncMethodVisitor` to implement a recursive method to replay annotation values as Listing 8.9 shows.

This method handles all annotation value types recursively. When creating the new non-synchronized method, the visitor applies all recorded annotations as seen in Listing 8.10.

This ensures that all annotations, including JMC's own `@JmcCheck` annotations, are preserved when synchronized methods are transformed.

8.4 Concurrent Feature Detection

As part of integrating JMC with large open-source projects like Apache Iceberg [2], it became necessary to understand what concurrency features a codebase uses. This information helps identify which JMC instrumentation visitors are needed and which features might be unsupported.

8.4.1 Implementation

JMC provides `JmcConcurrentDetector` and `JmcConcurrentVisitor` to detect usage of `java.util.concurrent` classes as shown in Listing 8.11.

The `JmcConcurrentVisitor` applies this detector to all methods in a class and aggregates the results. This can be seen in Listing 8.12.

```java
private void writeAnnotationValue(
        AnnotationVisitor annotationVisitor,
        String name,
        VisitorHelper.AnnotationValue value) {
    switch (value.type()) {
        case Primitive -> {
            annotationVisitor.visit(name,
                ((VisitorHelper.PrimitiveValue) value)
                    .getValue());
            break;
        }
        case Enum -> {
            ....
        }
        case Array -> {
            AnnotationVisitor arrayVisitor =
                annotationVisitor.visitArray(name);
            for (VisitorHelper.AnnotationValue v :
                    ((VisitorHelper.ArrayValue) value)
                        .getValues()) {
                writeAnnotationValue(
                    arrayVisitor, name, v);
            }
            break;
        }
        case Nested -> {
            ...
        }
    }
}
```

Listing 8.9: Recursive annotation value copying.

This tool proved invaluable during Iceberg integration, revealing which executors, locks, and concurrent collections were used, guiding the implementation of corresponding JMC support.

8.5 Integration Fixes

Integrating JMC with Apache Iceberg [3] revealed several edge cases where JMC's instrumentation conflicted with library code. Two specific issues required targeted fixes.

8.5.1 Listener.notify() False Positive

Iceberg's `Listener` class has a `notify()` [37] method that is not related to `Object.notify()`:

JMC's `JmcWaitNotifyVisitor` was incorrectly instrumenting this method because it only checked the method name. The fix, seen in Listing 8.13, was to add an owner check.

```java
private void addSyncMethod(
        VisitorHelper.MethodInfo methodInfo) {
    MethodVisitor newMv = cv.visitMethod(
        methodInfo.getNonSyncAccess(),
        methodInfo.getName(),
        methodInfo.getDescriptor(),
        methodInfo.getSignature(),
        methodInfo.getExceptions());

    // Copy all annotations
    for (VisitorHelper.AnnotationInfo ann
            : methodInfo.getAnnotations()) {
        AnnotationVisitor newMvAv =
            newMv.visitAnnotation(
                ann.getDescriptor(),
                ann.getVisibility());
        for (Map.Entry<String,
                VisitorHelper.AnnotationValue> e
                : ann.getValues().entrySet()) {
            writeAnnotationValue(newMvAv,
                e.getKey(), e.getValue());
        }
        newMvAv.visitEnd();
    }
    ...
}
```

Listing 8.10: Applying copied annotations to the new method.

8.5.2 Guava Joiner.join() vs Thread.join()

A more subtle issue arose with Guava's `Joiner` class [29]. This is shown in Listing 8.14.

JMC instruments `Thread.join()` to integrate with its scheduling. However, earlier we only checked the method name `"join"`, which meant JMC also instrumented Guava's `Joiner.join(Iterable<?>)` [29].

This resulted in a verification error, as seen in Listing 8.15, and the fix was to check both the method name and the owner type. JMC now uses runtime type checking as seen in Listing 8.16.

```java
public class JmcConcurrentDetector
        extends MethodVisitor {
    private final Set<String> detectedFeatures =
        new HashSet<>();

    @Override
    public void visitMethodInsn(int opcode,
            String owner, String name,
            String descriptor, boolean isInterface) {
        if (owner.startsWith(
                "java/util/concurrent")) {
            String feature =
                owner.replace("/", ".") + "." + name;
            detectedFeatures.add(feature);
        }
        super.visitMethodInsn(opcode, owner, name,
            descriptor, isInterface);
    }

    public Set<String> getDetectedFeatures() {
        return detectedFeatures;
    }
}
```

Listing 8.11: JmcConcurrentDetector for feature detection.

The `shouldInstrumentThreadCall()` method checks at runtime whether the object is actually a `Thread` instance as seen in Listing 8.17.

This approach handles all cases correctly: `Thread.join()` is instrumented, while `Joiner.join()` and any other `join()` methods are left unchanged.

Note: The `transient` keyword [52] specifies that a field should not be included during serialization. By contrast, `volatile` [53] fields enforce visibility and ordering guarantees under the Java Memory Model, ensuring that updates made by one thread are visible to others. In the current design, JMC treats all field accesses uniformly and does not distinguish between `transient` and `volatile` fields during instrumentation.

```java
public class JmcConcurrentVisitor
        extends ClassVisitor {
    private final List<JmcConcurrentDetector>
        methodDetectors = new ArrayList<>();

    @Override
    public MethodVisitor visitMethod(int access,
            String name, String desc,
            String signature, String[] exceptions) {
        MethodVisitor mv = super.visitMethod(
            access, name, desc,
            signature, exceptions);
        JmcConcurrentDetector detector =
            new JmcConcurrentDetector(
                Opcodes.ASM9, mv, className, name);
        methodDetectors.add(detector);
        return detector;
    }

    public Set<String> getAllDetectedFeatures() {
        Set<String> all = new HashSet<>();
        for (JmcConcurrentDetector detector
                : methodDetectors) {
            all.addAll(
                detector.getDetectedFeatures());
        }
        return all;
    }
}
```

Listing 8.12: JmcConcurrentVisitor aggregating detected features.

```java
@Override
public void visitMethodInsn(int opcode,
        String owner, String name,
        String descriptor, boolean isInterface) {
    // Only instrument wait/notify/notifyAll if
    // called on java.lang.Object
    if (opcode != Opcodes.INVOKEVIRTUAL
            || !owner.equals("java/lang/Object")) {
        super.visitMethodInsn(opcode, owner, name,
            descriptor, isInterface);
        return;
    }

    // Instrument Object.wait(), Object.notify(),
    // Object.notifyAll()
    // ...
}
```

Listing 8.13: Owner check to avoid false positive on wait().

```java
// Guava's Joiner.join()
String result = Joiner.on(",").join(items);
// Thread.join()
thread.join();
```

Listing 8.14: Guava Joiner.join() vs Thread.join() — name collision.

```
java.lang.VerifyError: Bad type on operand stack
Exception Details:
  Location:
    org/apache/iceberg/Schema.toString()
    Ljava/lang/String; @83: invokestatic
  Reason:
    Type 'java/lang/Iterable'
    (current frame, stack[5])
    is not assignable to integer
```

Listing 8.15: VerifyError from instrumenting Joiner.join() as Thread.join().

```java
@Override
public void visitMethodInsn(int opcode,
        String owner, String name,
        String descriptor, boolean isInterface) {
    if (name.equals("join")
            && opcode == Opcodes.INVOKEVIRTUAL
            && JOIN_DESCRIPTORS.contains(descriptor))
    {
        // Duplicate top of stack
        mv.visitInsn(Opcodes.DUP);

        // Check if it's actually a Thread
        mv.visitMethodInsn(Opcodes.INVOKESTATIC,
            "org/mpi_sws/jmc/runtime/JmcRuntimeUtils",
            "shouldInstrumentThreadCall",
            "(Ljava/lang/Object;)Z",
            false);

        ...
    }
}
```

Listing 8.16: Runtime type check for Thread.join() instrumentation.

```java
public static boolean shouldInstrumentThreadCall(
        Object obj) {
    return obj instanceof Thread;
}
```

Listing 8.17: Runtime Thread instance check.

Chapter 9

Evaluation

In this chapter, we evaluate the extensions to JMC presented in the preceding chapters by applying JMC to Apache Iceberg [2], an open-source table format for large-scale analytic datasets. Iceberg provides ACID transaction guarantees, schema evolution, time travel, and row-level operations for data stored in distributed file systems [54]. Its core library contains challenging patterns like static executor pools, optimistic concurrency control with CAS-based commits [14, 22, 12], manifest merging, `CompletableFuture`-based parallel task execution, and extensive use of lambdas, making it a demanding target for a concurrency testing tool.

We first demonstrate that JMC's extensions are correctly implemented through a suite of synthetic stress tests (Section 9.1). We then systematically test Iceberg's concurrency-critical features across three catalog implementations (Section 9.2), and report on the reproduction of three known concurrency issues (Section 9.3).

All experiments were conducted on an Apple MacBook Pro running macOS Sonoma 14.2, with Apple M1 Pro, 16GB of RAM.

9.1 Synthetic Stress Tests

To show that the extensions described in Chapters 3–8 are correctly implemented, we developed a suite of 17 synthetic stress tests. Each test targets a specific concurrency pattern or combination of patterns discussed in the preceding chapters. Table 9.1 summarizes the results.

All 17 tests pass under JMC, confirming that the extensions for static initialization, lambda expressions, executors and futures, `Object` class methods, and miscellaneous patterns are correctly implemented and do not interfere with each other. However, a brief note about these tests - these are not isolated tests, almost all of them heavily use `Executors` and `Futures` so as to mimic commercial scenarios as closely as possible. With this foundation established, we turn to evaluating JMC against a real-world application.

9.2 Testing Apache Iceberg with JMC

Apache Iceberg [2] is a widely adopted open table format for large-scale analytic datasets. Its architecture is organized into three layers: the metadata layer, the data layer, and the catalog [54].

#	Pattern Tested	Time (s)	
		1000 iter.	**10 000 iter.**
1	Static Init – Concurrent Access	0.426	2.717
2	Static Init – Nested Dependencies	0.430	2.889
3	Static Synchronized Methods	0.301	1.749
4	Interface Static Fields	0.055	0.168
5	Lambda / invokedynamic	0.643	4.745
6	ScheduledExecutorService	0.355	2.110
7	Executor Lifecycle / TrackExecutors	0.320	2.175
8	hashCode/equals + ConcurrentHashMap	0.067	0.174
9	wait/notify/notifyAll	0.491	3.502
10	toString infinite read	0.521	3.714
11	Enum skip instrumentation	0.180	0.733
12	Final fields skip	0.316	2.217
13	Read/Write instrumentation	0.462	3.212
14	Synchronized annotation preservation	0.194	1.275
15	Method name collisions	0.070	0.302
16	Volatile/Transient fields	0.415	3.133
17	Shared mutable state / data race	0.417	3.055

Table 9.1: Synthetic stress tests validating JMC's extensions.

The **metadata layer** forms the core of an Iceberg table's architecture. It is a tree structure composed of three file types: manifest files, manifest lists, and metadata files. This tree tracks data files and the operations that produced them, enabling features such as time travel and schema evolution[54].

The **data layer** stores the actual table data, consisting primarily of data files and delete files. These files form the leaves of the metadata tree. In production, the data layer is backed by distributed file systems (e.g., HDFS) or object storage (e.g., Amazon S3, Azure ADLS, Google Cloud Storage), enabling data lakehouse architectures built on scalable, low-cost storage[54].

The **catalog** serves as the entry point for all table interactions. It stores a pointer to the current metadata file for each table. The primary requirement for an Iceberg catalog is that it must support atomic operations for updating this metadata pointer, ensuring that all readers and writers observe a consistent table state[54].

Note: The Iceberg tests report average execution time; this value corresponds to the time per test.

9.2.1 Targeting the Metadata Layer

We began our exploration by targeting the metadata layer, validating the commit operation through the `InMemoryCatalog`. The `TestInMemoryCatalogJmc` test suite (located in `iceberg/core`) exercises the full commit path: multiple threads concurrently create data files with distinct records, append them via `FastAppend`, and verify that the final table contains all records with no duplicates or losses. Table 9.4 shows these results.

However, even with the relatively lightweight `InMemoryCatalog`, the full commit path tests proved difficult to scale beyond a small number of iterations, particularly for complex operations

Test class name	Jmc Tests count	Avg. Time		
		10 iter.	100 iter.	1000 iter.
TestInMemoryCatalogJmc	4	∼30 s	∼5 min	⏲
TestConcurrentSchemaMutationsJmc	4	∼0 s	∼20 s	∼4 min
TestExpireSnapshotsJmc	3	∼2 min	⏲	⏲
StaleSchemaOverwriteJmc	2	∼5 min	⏲	⏲
IcebergRowDeltaConflictJmc	3	∼1 min	∼10 min	⏲

Table 9.2: Metadata layer tests via InMemoryCatalog. ⏲ denotes a 20-minute timeout. Times are averages across runs.

like `OverwriteFiles` and `RowDelta`.

9.2.2 Targeting Base Classes and Infrastructure

We then shifted focus to testing the underlying base classes and internal infrastructure that Iceberg's higher-level operations are built upon. These tests, as seen in Table 9.3, target classes such as
`BaseMetastoreOperations`, `BaseMetastoreTableOperations`, `BaseTransaction`, and the `Tasks` parallel execution framework [3]. By operating at this level, the tests avoid the overhead of full commit paths while still exercising the concurrency-critical logic in commit status checking, metadata refresh, transaction state management, and parallel task coordination.

Test class name	Jmc Tests count	Avg. Time	
		1000 iter.	10 000 iter.
TestTasksFrameworkJmc	6	∼0 s	∼30 s
TestSimpleMetastoreJmc	8	∼5 s	∼30 s
TestTransactionsJmc	5	∼1 min	∼10 min
TestBaseMetastoreOperationsJmc	5	∼10 s	∼1 min
TestMetastoreTableOperationsJmc	5	∼20 s	∼8 min

Table 9.3: Base class and infrastructure tests. Times are averages across runs.

We noted that some tests from `TestTasksFrameworkJmc` did fail with assertion violations triggered by race conditions; however, upon further investigation these turned out to reflect expected behaviour in the `stopOnFailure` coordination logic rather than actual bugs. Few tests in `TestMetastoreTableOperationsJmc` fail with `UnsupportedOperationException`.

This indicated that the test-level mock implementations lacked the concrete functionality needed to exercise the full code paths. We concluded that while testing at this level was fast, it was insufficient for finding real concurrency bugs, as key methods were stubbed out and only available in concrete catalog implementations.

9.2.3 Catalog Implementations and CAS Mechanisms

Iceberg's ACID guarantees are implemented through optimistic concurrency control: each operation reads the current table metadata, builds a new metadata version, and attempts an atomic compare-and-swap (CAS) commit through the catalog. On conflict (`CommitFailedException`),

the operation refreshes its metadata and retries. The correctness of this mechanism depends on the catalog's CAS implementation and the retry logic in `SnapshotProducer.commit()` [54].

Different catalog backends implement the CAS differently. The `InMemoryCatalog` uses `ConcurrentHashMap.compute()` for its atomic update [14]. The JDBC catalog wraps the metadata pointer update in a SQL transaction with a version check against the given database engine [22]. The Hadoop catalog relies on atomic file system renames to swap the metadata pointer [12]. For the REST catalog, the CAS is performed via HTTP requests to the preferred server, which manages versioned references internally [4].

To integrate JMC with the JDBC catalog, we adapted one of Iceberg's existing concurrency tests for JDBC tables. Since these tests are heavier than `InMemoryCatalog` tests, we were only able to run them for 10 iterations within the timeout. For the Hadoop and REST catalogs, we wrote our own test suites. The REST catalog tests use Nessie [42], MinIO [38], and Testcontainers [50] to provide a full integration environment. Table 9.4 demonstrates the results.

Test class name	**Catalog**	**Jmc Tests count**	**Avg. Time (10 iter.)**
TestJdbcTableConcurrencyJmc	JDBC	1	∼4 min
TestHadoopTablesJmc	Hadoop	1	∼6 min
IcebergRESTCatalogConcurrencyJmc	REST	3	∼8 min

Table 9.4: Catalog integration tests. REST setup uses Nessie, MinIO, and Testcontainers. Times are averages across runs.

Since JMC currently instruments all field accesses (`GETFIELD`/`PUTFIELD`), it introduces instrumentation overhead that is especially visible with complex operations like `OverwriteFiles` and `RowDelta` in the `InMemoryCatalog`, and with append operations in the REST, Hadoop, and JDBC catalog integration tests.

9.3 Reproduction of Known Concurrency Issues

We used JMC to reproduce three known concurrency issues reported against Iceberg's REST catalog [4] implementation (via Nessie). These tests require a full integration environment with Nessie [42] and MinIO [38] containers managed by Testcontainers [50].

Issue #14093: Schema Lost on Concurrent Commit. [45] A race condition where `SnapshotProducer` captures table metadata in its constructor and uses it during commit, potentially overwriting concurrent schema changes with stale metadata. The test follows the exact scenario from the issue report: a background data append races with a schema evolution, and the final schema ID is checked for regression.

Issue #15001: Concurrent Commit Sequence Number Race. [44] When several processes attempt to commit simultaneously via the REST catalog, some of these commits fail. Instead of the expected retryable `CommitFailedException` [3], they throw a non-retryable `ValidationException` [3] related to sequence numbers. The test involves submitting three

concurrent append operations and then verifying whether any of them fail due to the sequence number validation error.

Issue #15008: Concurrent View Replacement. [6] When the same view change is applied twice concurrently, the server-side commit handler fails because `lastAddedSchemaId` is consumed on the first application and left null for the second. The test concurrently replaces a view with the same schema from two threads.

We record these as reproductions of previously reported issues, not new discoveries.

Chapter 10

Conclusion

This thesis presented a comprehensive set of extensions to JMC [25] that transform it from a research prototype into a practical model checker capable of systematically testing large-scale, real-world concurrent Java applications. We addressed fundamental instrumentation and runtime challenges like static initialization with synchronized reinitialization and nested dependency tracking, lambda expressions via `invokedynamic`, executor and future lifecycle management through `TrackExecutors`, native method handling for `Object` class methods, and support for enums, interfaces, and miscellaneous patterns encountered in production code. A custom JUnit 5 test engine provides seamless integration with standard Java development workflows.

We demonstrate the effectiveness of the extensions we made to JMC [25] through a series of synthetic stress tests. The successful integration with Apache Iceberg [3], a production-grade distributed storage system with complex code patterns including static executor pools, parallel I/O, and `CompletableFuture` chains, validates that JMC can operate on real-world codebases at scale. The test suite over Apache Iceberg [3] demonstrates JMC's capabilities in being able to handle Iceberg's full concurrency stack, including CAS-based commits, manifest merging, metadata validation, transaction replay and distributed locking.

These results demonstrate JMC's potential as a practical and viable tool for systematic concurrency testing of Java applications. Performance evaluation is outside the scope of this work, but remains an important direction for future optimization.

Bibliography

[1] Annotations in java. https://docs.oracle.com/javase/specs/jls/se7/html/jls-9.html#jls-9.6.

[2] Apache iceberg: Open table format for analytic datasets. https://iceberg.apache.org/.

[3] Apache iceberg: Open table format for analytic datasets. https://github.com/apache/iceberg.

[4] Apache iceberg: REST catalog. https://github.com/apache/iceberg/blob/2abac79fcae94b5ad039bd09f7235be191b0761e/core/src/main/java/org/apache/iceberg/rest/RESTCatalog.java#L47.

[5] ASM: An all purpose java bytecode manipulation and analysis framework. https://asm.ow2.io/.

[6] Concurrently re-applying the same changes to a view fails (#15008). https://github.com/apache/iceberg/issues/15008.

[7] Detailed initialization procedure. https://docs.oracle.com/javase/specs/jls/se17/html/jls-12.html#jls-12.4.2.

[8] Enums in the java language specification. https://docs.oracle.com/javase/specs/jls/se7/html/jls-8.html#jls-8.9.

[9] ExecutorService in java. https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/ExecutorService.html.

[10] Final fields in java. https://docs.oracle.com/javase/specs/jls/se17/html/jls-4.html#jls-4.12.4.

[11] Fray: A concurrency testing framework for java. https://github.com/cmu-pasta/fray.

[12] Hadoop catalog commit operation. https://github.com/apache/iceberg/blob/2abac79fcae94b5ad039bd09f7235be191b0761e/core/src/main/java/org/apache/iceberg/hadoop/HadoopTableOperations.java#L162.

[13] Hotspot VM intrinsics. https://github.com/openjdk/jdk/blob/master/src/hotspot/share/classfile/vmIntrinsics.hpp.

[14] InMemory catalog commit operation. https://github.com/apache/iceberg/blob/2abac79fcae94b5ad039bd09f7235be191b0761e/core/src/main/java/org/apache/iceberg/inmemory/InMemoryCatalog.java#L408-L422.

[15] Instrumentation in java.lang.instrument. https://docs.oracle.com/javase/8/docs/api/java/lang/instrument/package-summary.html.

[16] IntrinsicCandidate annotation. https://github.com/openjdk/jdk/blob/master/src/java.base/share/classes/jdk/internal/vm/annotation/IntrinsicCandidate.java.

[17] Invokedynamic instruction. https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.10.1.9.invokedynamic.

[18] Invoking methods with reflection. https://docs.oracle.com/javase/tutorial/reflect/member/methodInvocation.html.

[19] Java agent. https://docs.oracle.com/en/java/javase/21/docs/api/java.instrument/java/lang/instrument/package-summary.html.

[20] Java pathfinder. https://github.com/javapathfinder/jpf-core/wiki.

[21] java.lang.object. https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html.

[22] JDBC catalog commit operation. https://github.com/apache/iceberg/blob/2abac79fcae94b5ad039bd09f7235be191b0761e/core/src/main/java/org/apache/iceberg/jdbc/JdbcTableOperations.java#L110-L120.

[23] JMC annotations. https://jmc.mpi-sws.org/user_guide/docs/usage_guide/writingtests/.

[24] JMC API documentation. https://jmc.mpi-sws.org/api_docs/.

[25] JMC github. https://github.com/mpi-sws-rse/jmc.

[26] JMC instrumentation. https://jmc.mpi-sws.org/user_guide/docs/development/instrumentation/.

[27] JMC runtime. https://jmc.mpi-sws.org/user_guide/docs/development/runtime/.

[28] JMC user guide and development documentation. https://jmc.mpi-sws.org/user_guide/docs/development/.

[29] Joiner class in guava. https://guava.dev/releases/31.1-jre/api/docs/com/google/common/base/Joiner.html.

[30] JUnit 5 user guide. https://junit.org/junit5/docs/current/user-guide/.

[31] JUnit5 test engine. https://docs.junit.org/6.0.3/advanced-topics/engines.html.

[32] JVM loading, linking, and initializing. https://docs.oracle.com/javase/specs/jvms/se16/html/jvms-5.html.

[33] JVM special initialization. https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-5.html#jvms-5.5.

[34] JVM verification of class files. https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.10.

[35] Lambda metafactory in java. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/invoke/LambdaMetafactory.html.

[36] Lincheck: A framework for testing concurrent data structures. https://github.com/JetBrains/lincheck.

[37] Listener class in apache iceberg. https://github.com/apache/iceberg/blob/149cc464f9b7df800cc5718af725983473819504/api/src/main/java/org/apache/iceberg/events/Listener.java#L23.

[38] MinIO: S3 compatible object store. https://www.min.io/.

[39] Mockito framework. https://site.mockito.org/.

[40] Native method stacks – JVM. https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-2.html#jvms-2.5.6.

[41] Object creation – java language specification. https://docs.oracle.com/javase/specs/jls/se17/html/jls-4.html#jls-4.3.1.

[42] Project nessie: Transactional catalog for data lakes with git-like semantics. https://projectnessie.org/.

[43] Registering a TestEngine. https://docs.junit.org/6.0.3/advanced-topics/engines.html#registration.

[44] REST catalog concurrent commit race condition (#15001). https://github.com/apache/iceberg/issues/15001.

[45] Snapshot producer overwrites newer schema changes with stale metadata (#14093). https://github.com/apache/iceberg/issues/14093.

[46] Spring framework reference documentation. https://docs.spring.io/spring-framework/reference/.

[47] Static classes, fields, methods. https://docs.oracle.com/javase/specs/jls/se17/html/jls-8.html.

[48] Streams in java. https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html.

[49] Synchronization methods – java language specification. https://docs.oracle.com/javase/specs/jls/se17/html/jls-17.html#jls-17.2.

[50] Testcontainers for java. https://www.testcontainers.org/.

[51] Thread in java. https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html.

[52] Transient field in java. https://docs.oracle.com/javase/specs/jls/se7/html/jls-8.html#jls-8.3.1.3.

[53] Volatile field in java. https://docs.oracle.com/javase/specs/jls/se7/html/jls-8.html#jls-8.3.1.4.

[54] *Apache Iceberg: The Definitive Guide*. 2024.

[55] Patrice Godefroid, Nils Klarlund, and Koushik Sen. DART: Directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 213–223, Chicago, IL, USA, 2005. Association for Computing Machinery.

[56] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2012.

[57] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Professional, 2002.

[58] Shan Lu, Soyeon Park, Eunsoo Seo, and Yuanyuan Zhou. Learning from mistakes: A comprehensive study on real world concurrency bug characteristics. In *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XIII)*, pages 329–339, Seattle, WA, USA, 2008. Association for Computing Machinery.